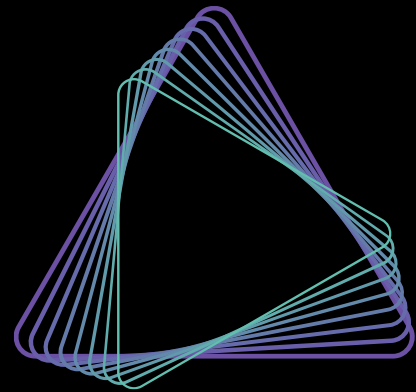




AZENTA 安升达
LIFE SCIENCES

生信必备-Python及其在NGS 数据分析中的应用

Han. Ping
NGS-BI-MED

2024年/8月/23日

学习流程



PYTHON介绍

为什么学习python

01

为什么学习Python

1. 简单易学

Python被认为是一种简单易学的编程语言，语法和结构设计非常清洗和直观，与自然语言比较接近，因此对初学者来说比较容易上手。

2. 广泛引用

Python在科学计算，数据分析，人工智能，机器学习等领域都有广泛的应用。在生物信息学中，研究员也常用Python来处理和分析大规模的生物数据。

3. 强大的生态系统

Python拥有丰富而强大的第三方库和工具，例如Numpy, Pandas, Matplotlib等，这些库使数据处理，分析和可视化变得更加高效和简单。

4. 开放性和社区支持

Python是开源的，拥有庞大的社区支持和活跃的开发社区。这意味着你可以轻松地找到解决问题的方法，学习资源和社区讨论。

Python在NGS数据分析中的优势

Python在NGS（下一代测序）数据分析中有许多优势，主要包括其丰富的库和工具、灵活性、易用性以及强大的社区支持。



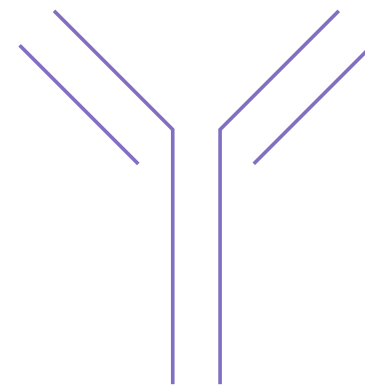
丰富的库和工具

提供处理生物序列数据的工具，方便解析各种NGS数据文件，个性化图表图形数据可视化。



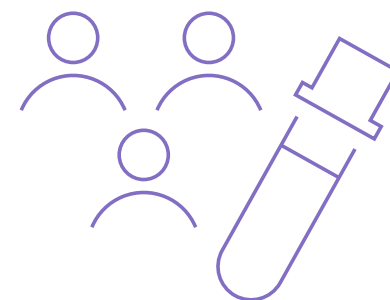
易用性和可读性

语法简洁易懂，不需要深厚的编程背景。



高效的数据处理和分析

Python与Pandas、NumPy等库结合，使得大规模数据处理和分析变得高效。



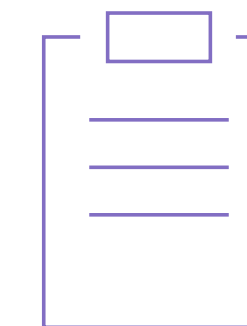
灵活性

可以与其他编程语言（C，C++，R）进行无缝集成。调用其他高效的代码，使数据处理分析更灵活。



可扩展性和自动化

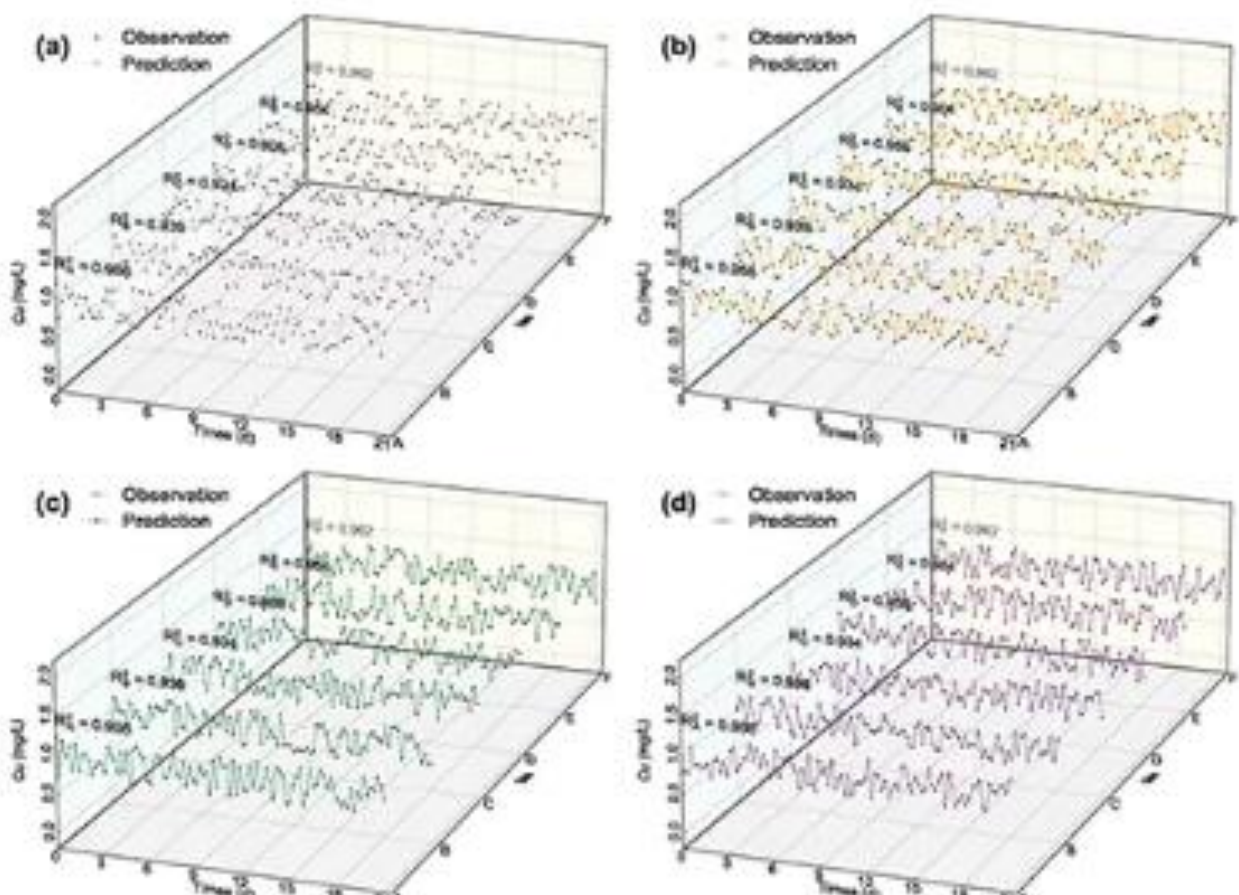
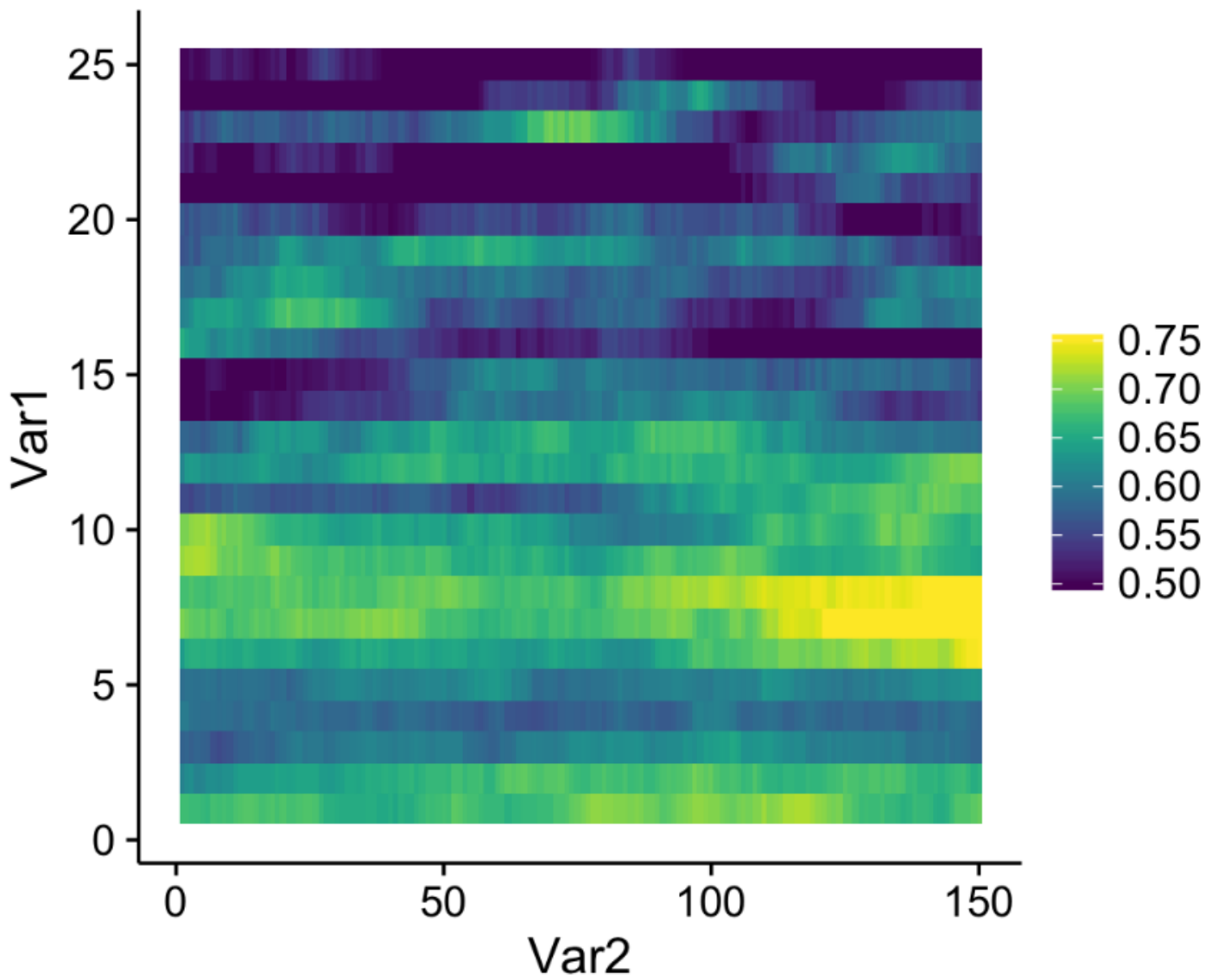
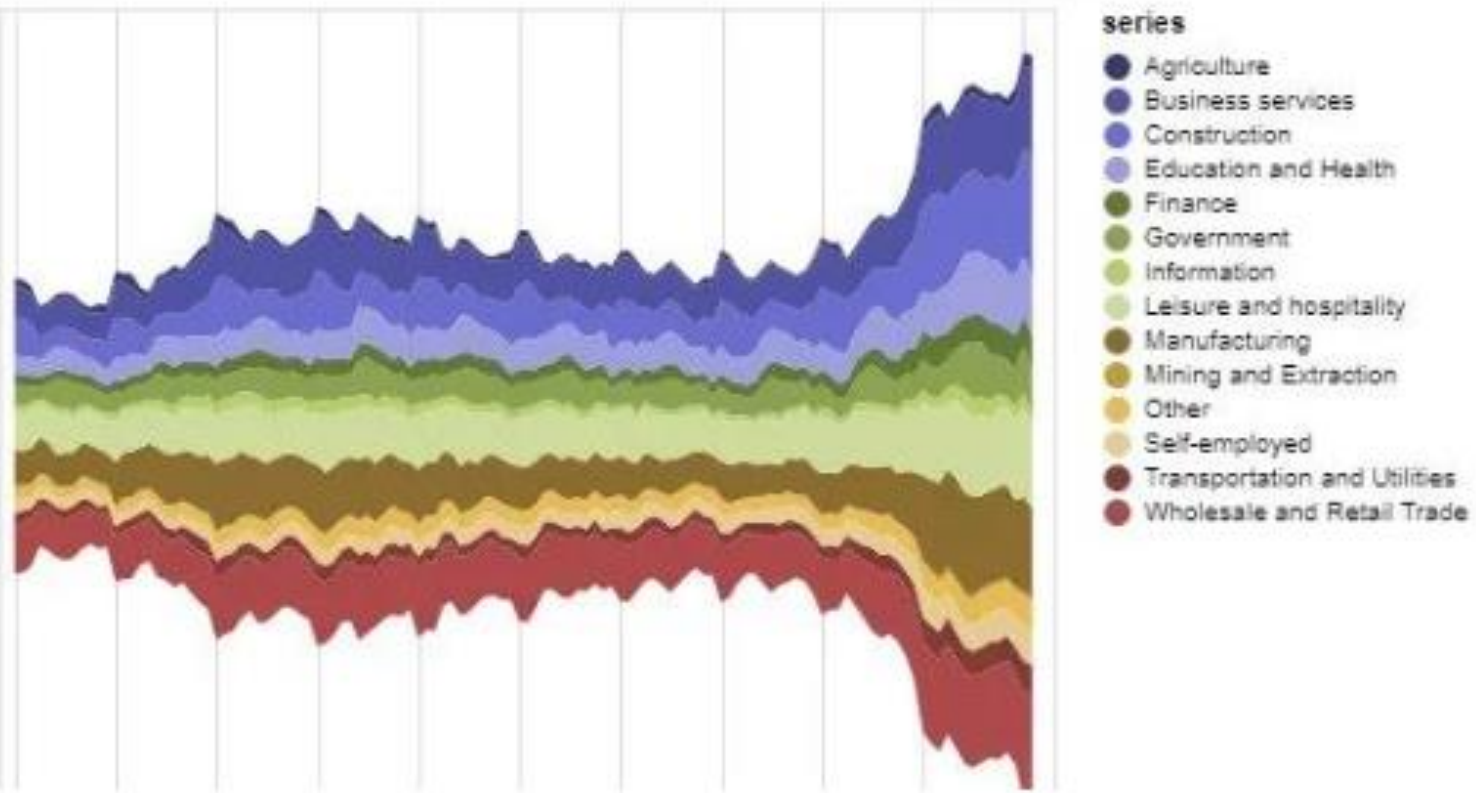
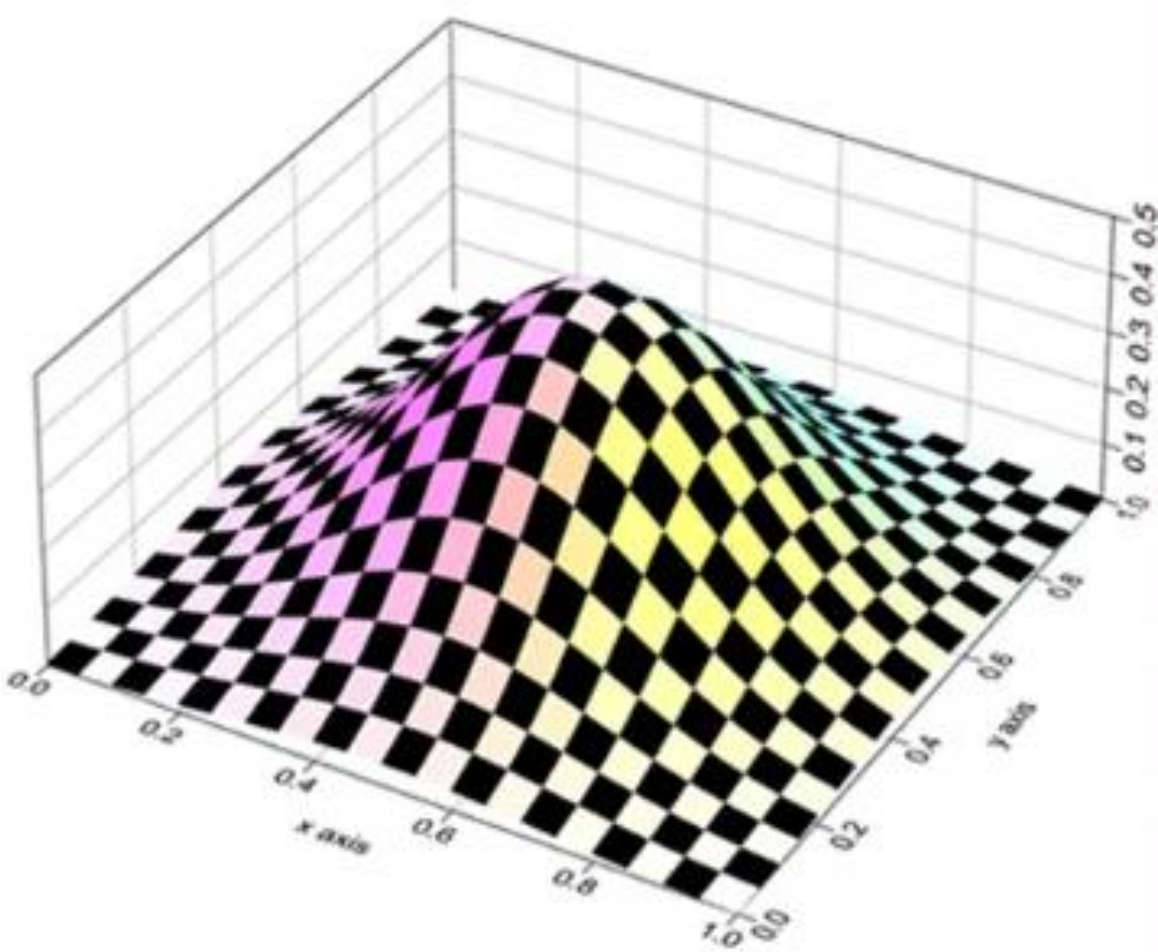
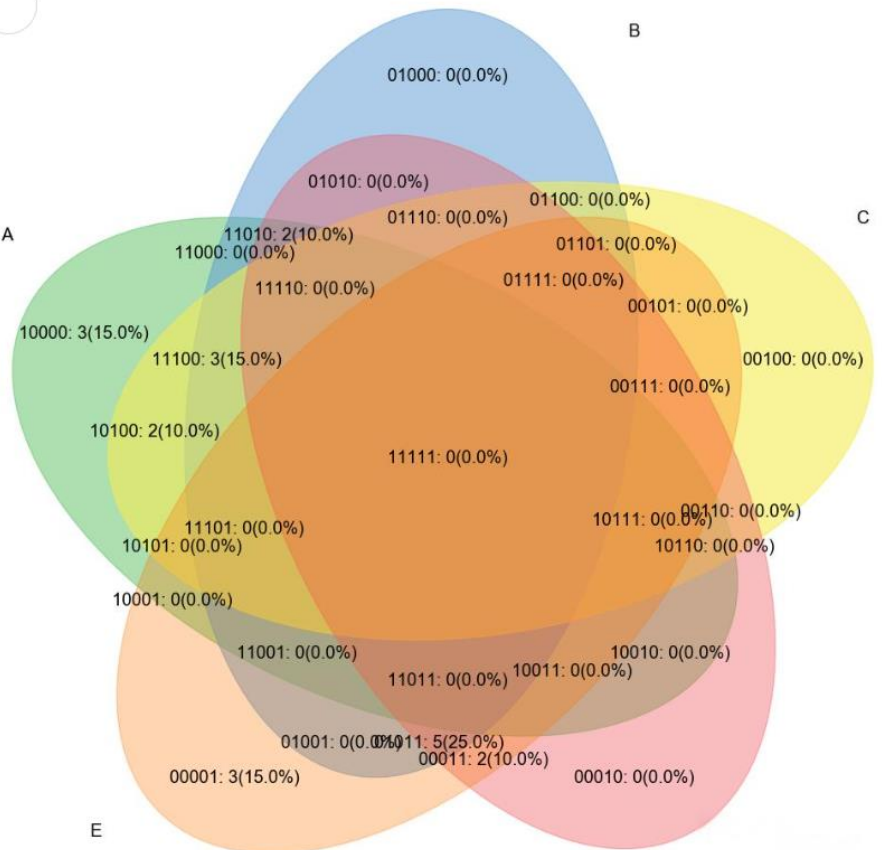
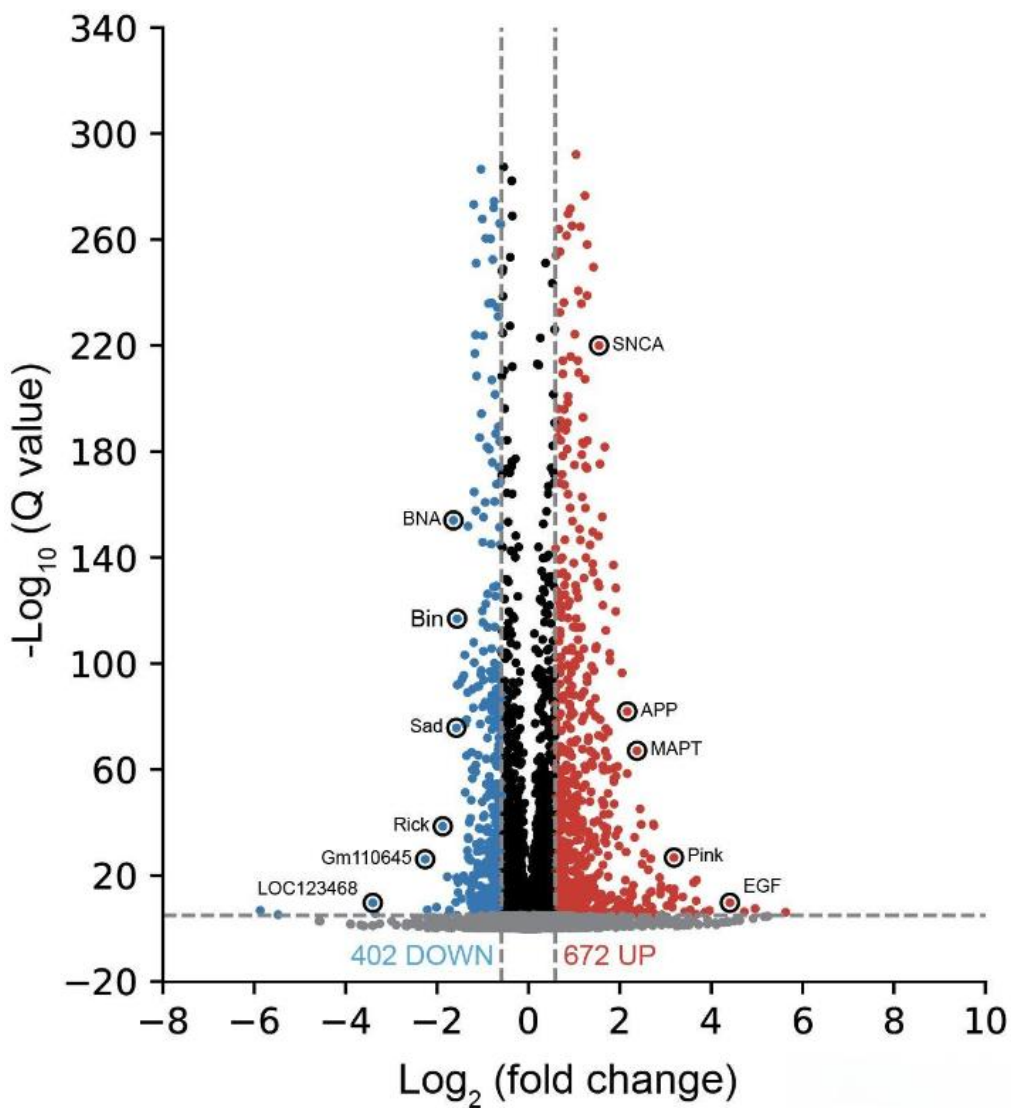
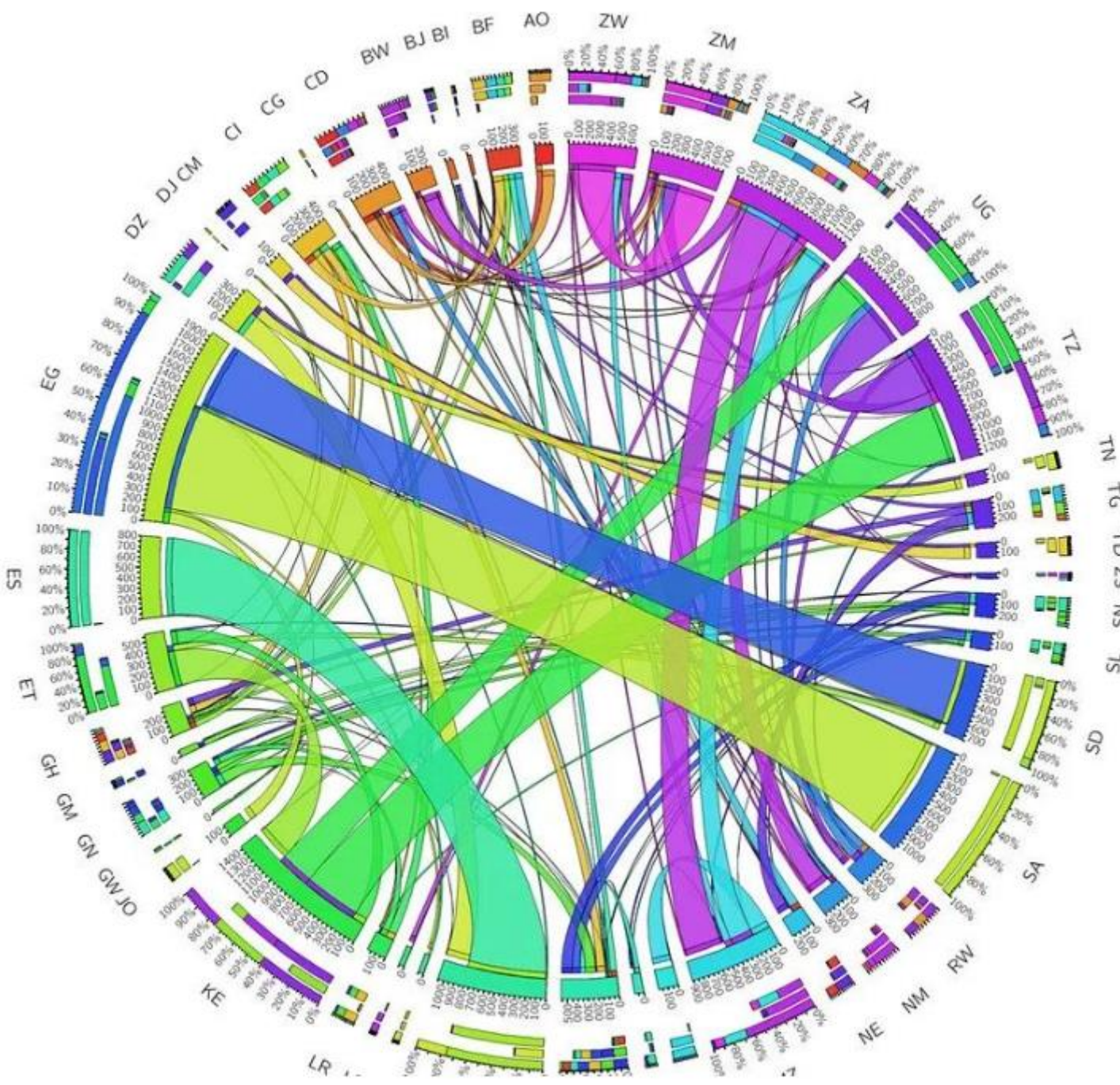
Python脚本可以轻松地自动化重复性的分析任务，并扩展适应不断变化的需求和新技术。



兼容性

Python可以与许多生物信息学工具和平台（如GATK, Samtools, BEDtools）进行兼容和集成。

Python在NGS数据分析中的优势



NGS数据分析实战问题

背景

作为一名生物信息学研究员，您正在研究一种新发现的生物基因组。您手头上有一个名为 ‘example.gb’ 的 GenBank 文件，其中包含了该生物的完整基因组信息。为了进一步分析该生物的基因组特性，您决定提取所有编码区（CDS）的基因序列，并计算每个基因的 GC 含量。**GC 含量（碱基对 G 和 C 在 DNA 序列中所占的比例）是判断基因组特性的重要指标之一，可能与基因的稳定性、功能和进化有关。**

研究任务

- 提取 CDS 序列：从 GenBank 文件中提取所有的编码区（CDS）序列。
- 计算 GC 含量：计算每个 CDS 序列的 GC 含量。
- 绘制 GC 含量分布图：生成一个柱状图，展示所有 CDS 序列的 GC 含量分布情况。

预期结果

使用python对该生物进行快速分析，更好地理解该生物基因组的组成和功能，并为进一步的生物学研究提供数据支持。

Python的下载安装

- Python官方 : <https://www.python.org/>
- Python官方文档下载地址: www.python.org/doc/
- Python的安装:
 - Linux or Unix/Mac OS: 自带python
 - Windows: 安装并设置环境变量
- Python的推荐IDE（集成开发环境）：PyCharm, Visual Studio Code, Jupyter notebook

Jupyter Notebooks

Jupyter Notebooks一般被用于数据专家，特别是数据分析师，数据科学家。根据[Kaggle2022年调查结果](#)。Jupyter Notebooks成为最受欢迎的数据科学IDE，使用率达到80%。

安装方法

```
>>pip install jupyterlab
```

启动

```
>>jupyter lab
```

```
>>jupyter notebook
```

Jupyter登入



输入密码 **12345678** 登入服务器

>>jupyter notebook # 启动jupyter notebook

```
ServerApp] Jupyter Server 2.14.1 is running at:
ServerApp] http://172.18.4.195:8889/tree
ServerApp] http://127.0.0.1:8889/tree
```

将该网址替换成公网网址，再浏览器中访问该地址

http://120.55.69.62:8889

基础语法与数据类型

Basic syntax and data types

02

基础语法

执行python的规则

在命令行直接写入执行（命令行模式）

```
(base) crispr001@iZbp18758pdxsp0rsgmsuhZ:~$ python
Python 3.12.2 | packaged by conda-forge | (main, Feb 16 2024, 20:50:58) [GCC 12.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("hello, world!")
hello, world!
>>>
```

或者可以创建一个已.py为后缀的python文件,在命令行中运行这个文件（脚本式编程）

```
print("hello, world!")
```

```
(base) crispr001@iZbp18758pdxsp0rsgmsuhZ:~$ python hello.py
hello, world!
```


基础语法

Python缩进

- Python与其他语言的最大区别就是，使用缩进来指示代码块。
- 缩进的空白数是可变的，但是所有代码块语句必须包含相同的缩进空白数量，这个必须严格执行。

如果缺少缩进，会引起语法错误：

缩进语法错误

```
if True:
print("True")
if True:
print("True")
```

```
Cell In[44], line 2
    print("True")
    ^
IndentationError: expected an indented block after 'if' statement on line 1
```

同一代码中需要使用相同的空格数

```
if 5 > 2 :
    print("True")
else:
    print("False")
```

基础语法

Python注释

1. 单行注释: 使用 # 符号。从 # 开始到该行结束的所有内容都是注释。

这是一个单行注释

print("Hello, World!") # 这也是一个单行注释

多行注释: 使用三个单引号 ''' 或三个双引号 """ 包围。

'''

这是一个多行注释

可以跨越多行

非常适合长篇解释

'''

基础语法

变量

变量是存储数据值的容器，变量可以被视作内存中一个预留位置的名称，在这个位置，可以存储各种类型的信息，如数字、文字或更复杂的数据类型。每个变量都会有一个独一无二的名称，程序通过这个名称对其进行访问或修改。Python中的变量不需要声明类型，是在赋值时创建的：

```
Seq = 'ATCG' # 字符串赋值
```

```
Lenth = 20 # 整数赋值
```

```
Rate = 3.5 # 浮点数赋值
```

```
Seq, Lenth, Rate = "ATCG", 20, 3.5 # 一行代码同时给多个变量赋值
```

变量可以在任何时候更改其类型：

```
x = str(20) # x will be '20'
```

```
y = int(20) # y will be 20
```

```
z = float(20) # z will be 20.0
```

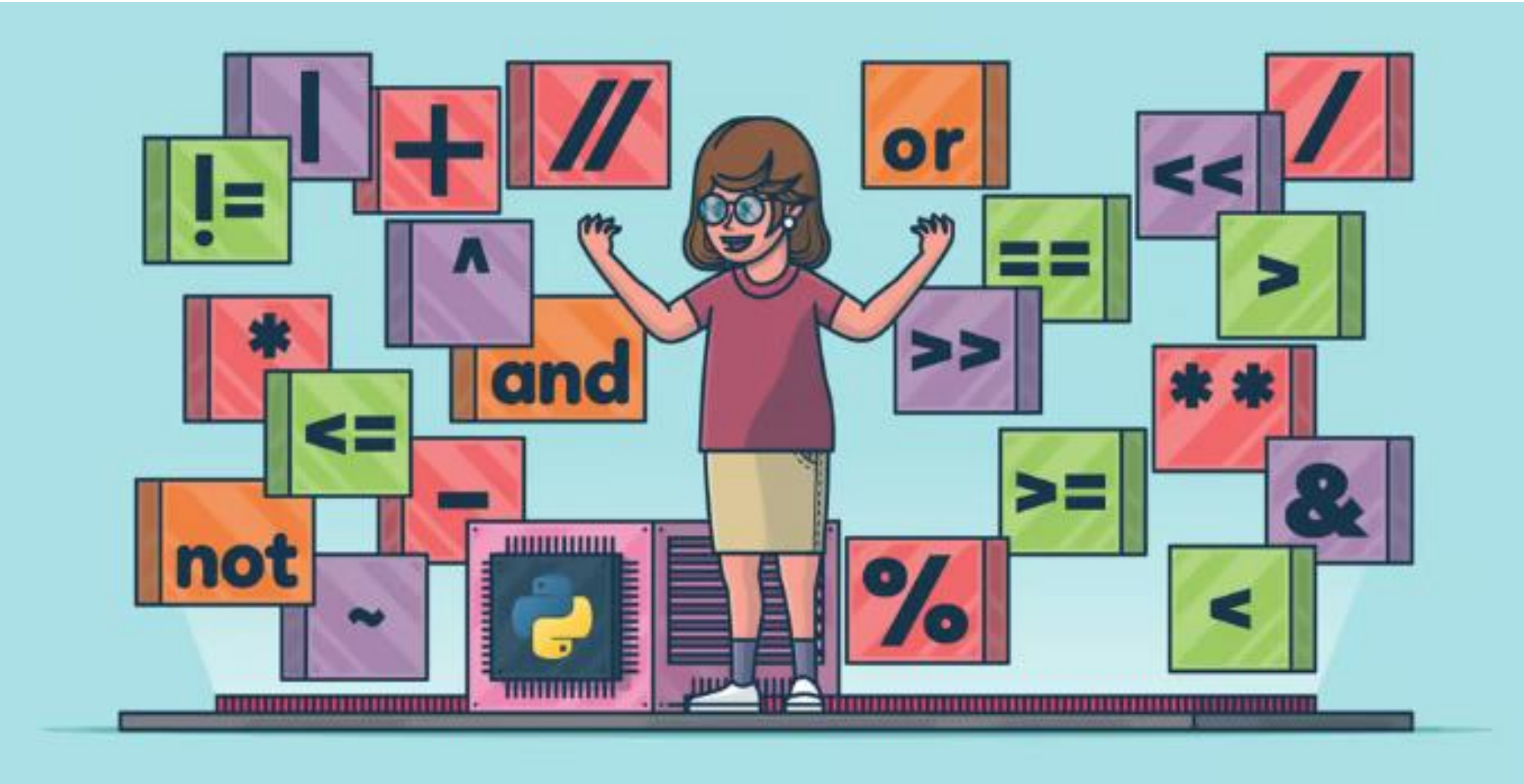

数据类型

Python提供了一些内置的数据类型，这些数据类型为python提供灵活的数据处理能力。

<code>var = "Sequence"</code>	<code># 字符串</code>
<code>var = 20</code>	<code># 整数型</code>
<code>var = 20.5</code>	<code># 浮点数</code>
<code>var = ["CDS", "exon", "intron"]</code>	<code># 列表</code>
<code>var = ("CDS", "exon", "intron")</code>	<code># 元组</code>
<code>var = range(6)</code>	<code># 范围</code>
<code>var = {"gene" : "TP53", "num" : 36}</code>	<code># 字典</code>
<code>var = {"CDS", "exon", "intron"}</code>	<code># 集合</code>
<code>var = True</code>	<code># 布尔</code>

Python运算（数字的操作）

类型	表示方式	示例
整数 (int)	没有小数部分的数，可以是正式负数和零	<code>a=10</code> <code>b=-5</code> <code>c=0</code>
浮点数 (float)	表示小数类型	<code>x=3.14</code> <code>y=0.0</code>
复数 (complex)	表示在一维实数轴上无法表示的数, 复数包括一个实部和一个虚部	<code>c1 = 3 + 4j</code>
布尔类型 (bool)	表示真 (True) 和假 (False) ，实际是整数的子类型， ‘True’ 等价于 ‘1’ ， ‘False’ 等价于 ‘0’	<code>t= True</code> <code>f= False</code>



Python运算



Python 提供了多种运算符，用于执行算术、赋值、比较、逻辑等操作。以下是一些常用运算符的示例，包括算术运算符、比较运算符、赋值运算符、逻辑运算符等。

类型

运算符

算术运算符	加(+),减(-),乘(),除(/), 取模(%), 幂(*), 取整除(//)
比较(关系)运算符	等于(==), 不等于(!=), 大于(>), 小于(<), 大于等于(>=), 小于等于(<=)
赋值运算符	赋值运算(=),加法赋值(+ =),减法赋值(- =),乘法赋值(=),除法赋值(/ =),取模赋值(% =),幂赋值(* =),取整除赋值(// =),海象运算符(:=)
逻辑(布尔)运算符	and, or, not
位运算符	按位与运算符(&),按位或运算符(
成员运算符	in, not in (指定的序列中是否找到值)
身份运算符	is, is not (用于比较两个对象的存储单元,判断两个标识符是不是引用自同一个对象)

Python运算

练习1

生信工程师小明收到一份测序数据，他已知道基因组的长度以及基因组的实际覆盖长度，现需要计算该测序数据的覆盖度情况，检查是否符合标准（80%）？

定义基因组长度和覆盖长度变量

```
genome_length = 1000 # 基因组总长度  
covered_length = 850 # 实际覆盖的长度
```

计算覆盖度

```
coverage = (covered_length / genome_length) * 100  
print(f"覆盖度: {coverage}%")
```

结果展示：

覆盖度： 85.0%

Python运算

练习2

生信工程师小明收到一份测序数据，他已统计出所有read的测序深度并输出一个列表类型的统计结果，现在他需要统计平均测序深度？

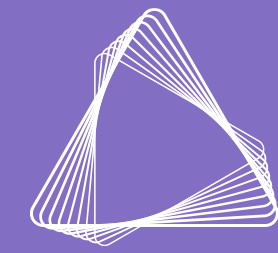
```
read_depths = [30, 45, 28, 50, 65, 42, 38, 55, 60, 48]
```

计算平均测序深度

```
average_depth = sum(read_depths) / len(read_depths)
print(f"平均测序深度: {average_depth}")
```

结果展示：

平均测序深度： 46.1



AZENTA 安升达
LIFE SCIENCES

‘sum’ 和 ‘len’ 为python中常用的两个内置函数，用于计算序列对象（如列表，元组，集合等）的总和和总长。

sum(s)返回对象中所有元素的总和，示例：

python

```
numbers = [1, 2, 3, 4, 5]
total = sum(numbers)
print(total) # 输出: 15
```

len(s)返回序列中元素的个数，示例：

python

```
my_string = "Hello, World!"
length = len(my_string)
print(length) # 输出: 13
```

Python运算

练习3

生信工程师小明收到一段DNA序列，他现在需要统计这段序列的GC含量？是否符合标准（40-60%）？

定义DNA序列变量

```
seq = "ATGCGCGATCGATCGCGATCGATCGCGCGATATCGCGCGCGCGAT"
```

计算GC含量

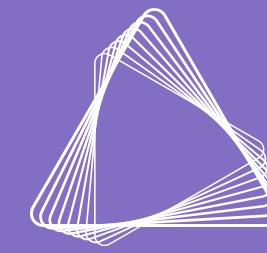
```
gc_count = sequence.count('G') + sequence.count('C' )
```

```
gc_content = (gc_count / len(sequence)) * 100
```

```
print(f"GC 含量: {gc_content:.2f}%")
```

结果展示：

GC 含量： 64.44%



AZENTA 安升达
LIFE SCIENCES

在python中， 'count' 是字符串、列表和元组等序列对象的方法之一，用于统计指定元素在序列中出现的次数。

1.对于字符串（str）：

python

```
my_string = "banana"
count_a = my_string.count('a')
print(count_a) # 输出：3，因为'a'在字符串中出现了3次
```

2.对于列表（list）或元组（tuple）

python

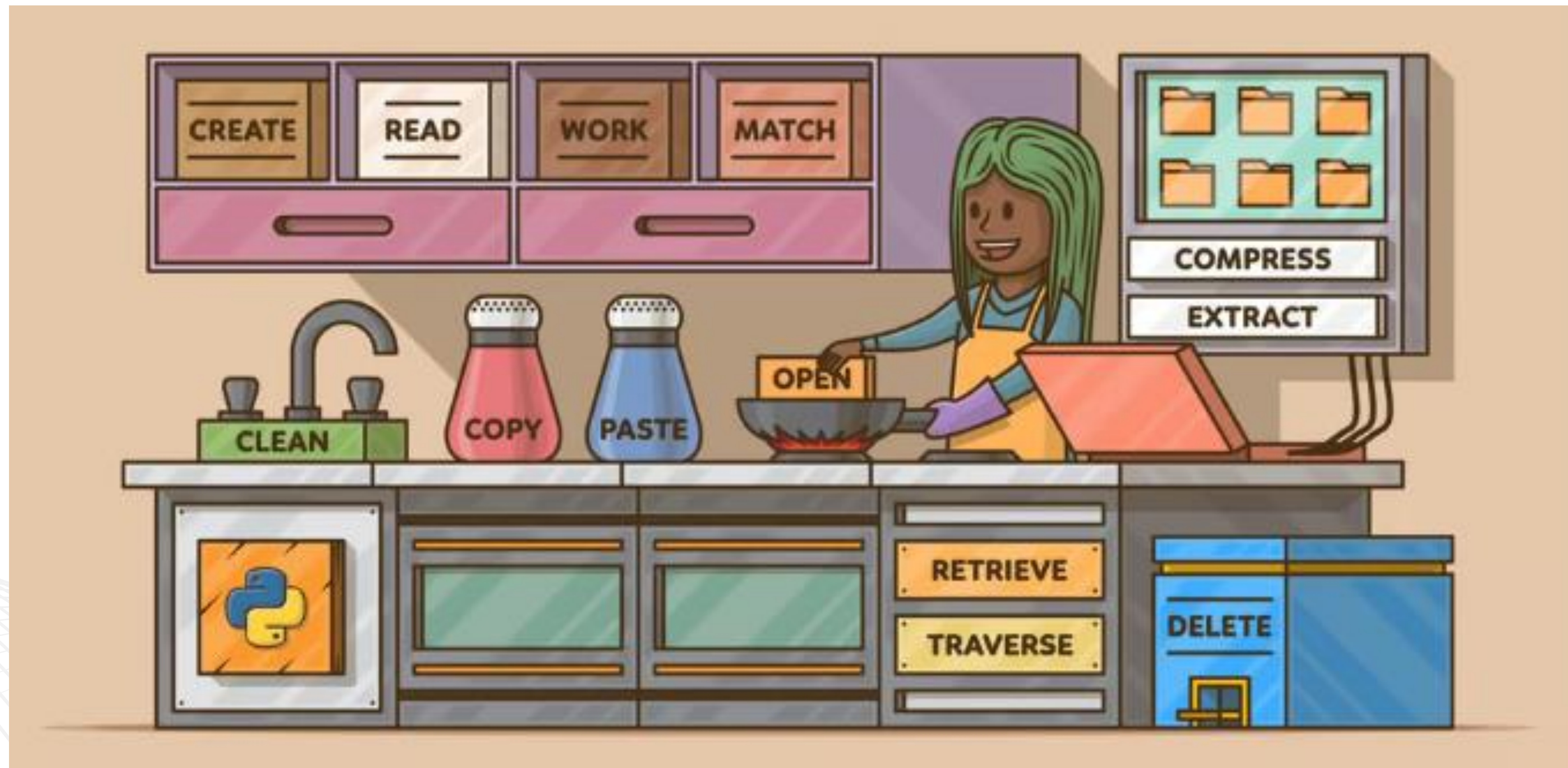
```
my_list = [1, 2, 3, 2, 4, 2, 5]
count_2 = my_list.count(2)
print(count_2) # 输出：3，因为数字2在列表中出现了3次
```

.2f: 用于指定浮点数的输出格式

- 'f' 表示格式化为浮点数
- '.2' 表示保留小数点后两位

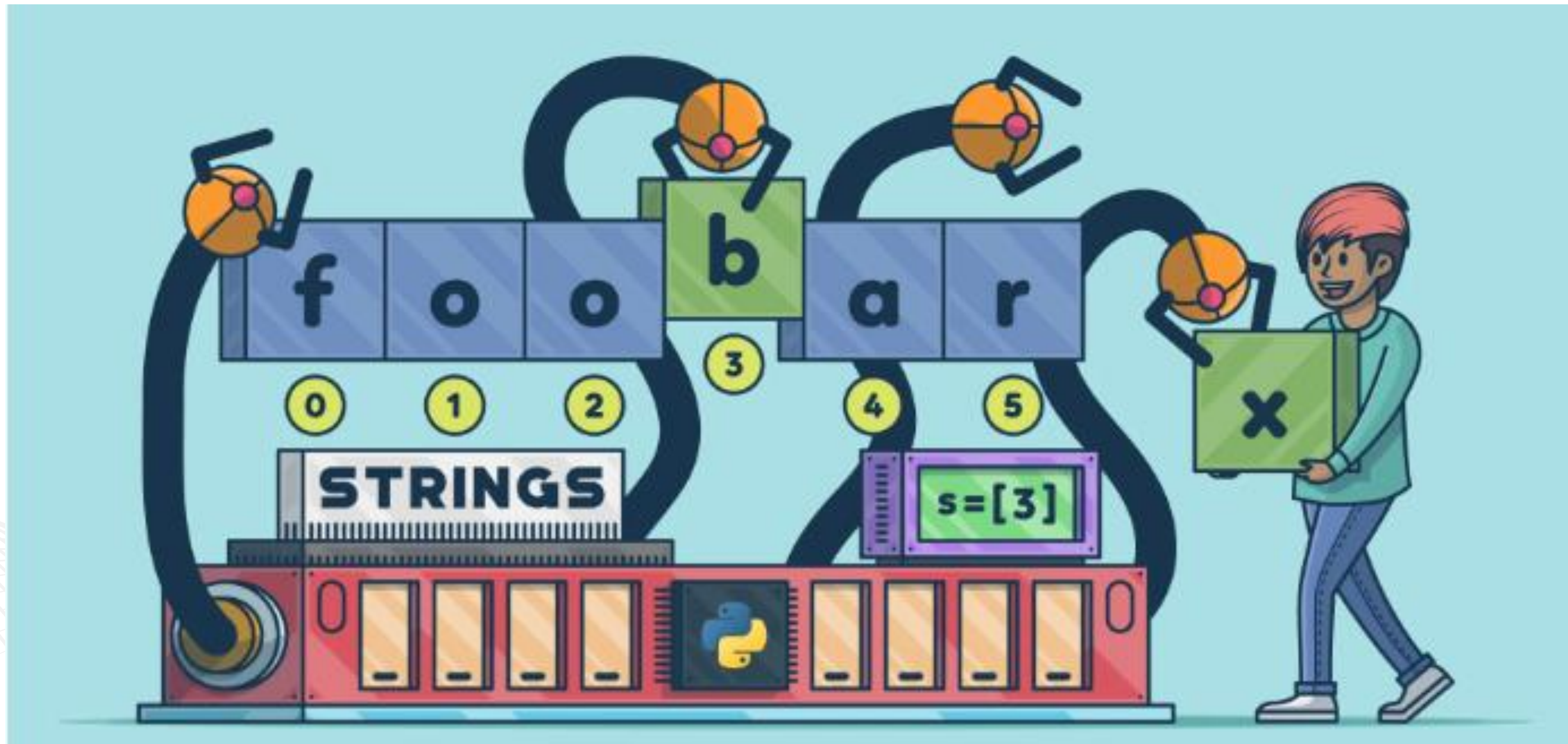
Python文本处理

Python不仅能够对数字进行处理，Python 同样能够应对从基本文本操作到复杂文本分析的各种需求。Python文本处理对NGS数据分析至关重要，因为大多数NGS数据都是文本类型。



字符串的操作

字符串（String）是由数字、字母、下划线组成的字符数据类型。它是编程语言中表示文本的数据类型。掌握字符串的操作，我们可以高效地处理和操作文本数据。

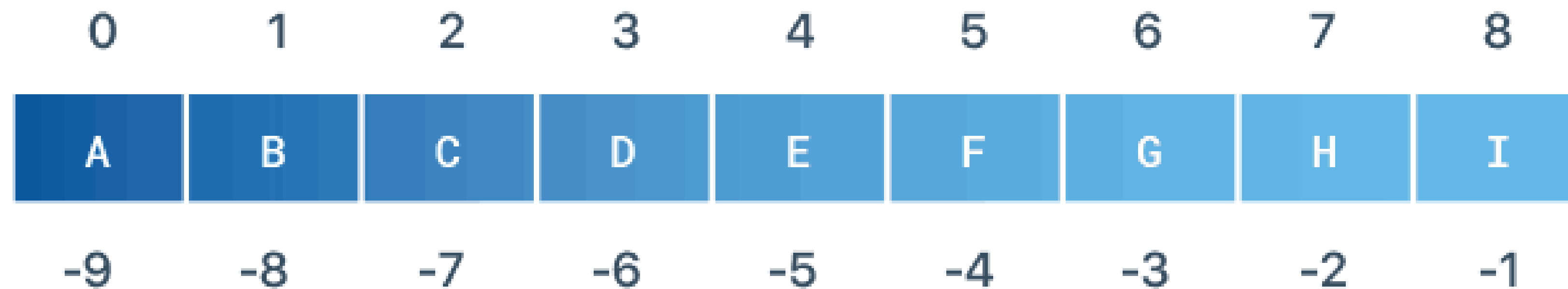


字符串的操作

字符串是有序序列，我们可以通过字符串的索引来访问特定位置的字符。

两种索引取值顺序：

- 字符串的正序索引通常从0开始，即第一个字符的索引为0，第二个字符的索引为1，以此类推。
- 字符串的倒序索引通常从-1开始，即第一个字符索引为-1，第二个字符的索引为-2，依次类推。



字符串的操作

索引取值的使用方法：

- 变量名[字符位置]
- 变量名[起始位置: 终止位置: 步长 (默认1)]
- 变量名[起始位置:] # 从起始位置到末尾
- 变量名[: 终止位置] # 从开头到终止位置

示例：

String = "ABCDEFGHI"

String[0] # 取值为A

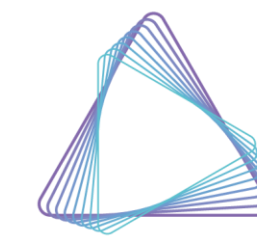
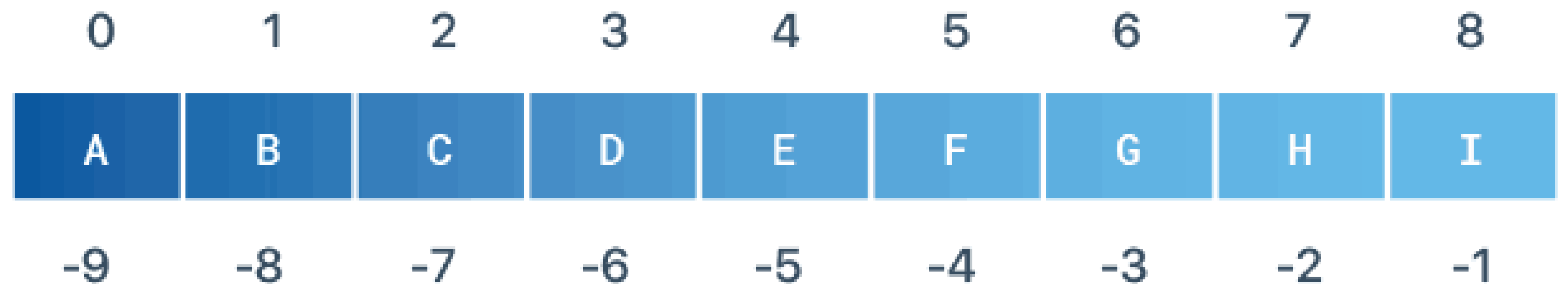
String[-1] # 取值为I

String[0:3] # 取值为ABC

String[:-6] # 取值为ABC

String[-3:] # 取值为GHI

String[::2] # 取值为ACEGI



AZENTA 安升达
LIFE SCIENCES

字符串的操作

练习4

生信工程师小明收到一条序列 ‘**ATCGATCGACTCGCACTCG**’, 现需要他使用字符串的操作方法快速得到该序列的反向序列？

```
Seq = 'ATCGATCGACTCGCACTCG'  
Reversed_seq = Seq[::-1]  
print(f"反向序列为: {Reversed_seq}")
```

结果展示：

反向序列为: GCTCACGCTCAGCTAGCTA

`[::-1]` 表示从头到尾以步长-1 进行切片，即从最后一个元素逐步向前移动一个位置直到第一个元素，从而实现反向。

字符串的操作

字符串还有许多其他的操作，列如字符串拼接，拆分字符串。

```
Seq1="Hello"
```

```
Seq2=","
```

```
Seq3="World"
```

```
Merge=Seq1 + Seq2 +Seq3
```

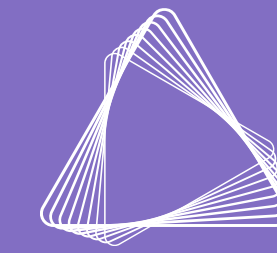
```
print(Merge)    # Hello,World
```

```
Words = Merge.split(',')
```

```
print(Words)    # ['Hello', 'World']
```

```
New = ','.join(Words)
```

```
print(New)      # Hello,World
```



Python中， 'split' 和 'join' 是字符串处理两个重要用法。 'split' 用于将字符串按照指定的分隔符分割成多个字符串，并返回一个包含这些字符串的列表。

```
str.split(sep=None, maxsplit=-1)
```

"sep" :指定分隔符

"maxsplit" : 指定最大分割次数

'join' 则用于将一个序列中多个字符串连接成一个字符串，并在每个字符串之间插入指定的分隔符。

```
str.join(iterable)
```

"iterable" :一个可迭代对象（列表，元组等）

练习5

我们现在有一段DNA序列

1. 序列长度?
2. 第50个核苷酸是什么?
3. 提取前10个核苷酸序列
4. 统计“AGCT”出现的次数

**ATGGCAGTGTGCATCGA
TGCTAGCTAGCTAGCTA
GCTAGCTGACTAGCTAG
CTAGCTAGCTAGCTAGC
TAATGC...**

练习5

1. 序列长度?
2. 第50个核苷酸是什么?
3. 提取前10个核苷酸序列
4. 统计“AGCT”出现的次数

```
Seq = 'ATGGCAGTGTGCATCGATGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAGCTAATGC'  
Length = len(Seq)  
nt = Seq[51]  
trimer = Seq[:10]  
counts = Seq.count("AGCT")
```

```
print(Length)    # 74  
print(nt)        # C  
print(trimer)    # ATGGCAGTGT  
print(counts)    # 11
```

list(列表) Tuple (元组) Set(集合) 区别

[] 定义
mylist = [1, 2, 3]
元素可重复
支持增删改查
有序，可变

() 定义
mytuple=(1,2,3)
元素可重复
支持增删改查
有序，不可变

{ } 定义
Myset={1, 2, 3}
元素不重复
支持增删
无序，不可变

总结:

- **List** 适合有序、重复元素、需要频繁修改的场景。
- **Tuple** 适合不变的、有序、重复元素的场景，尤其是当需要保证数据安全性时。
- **Set** 适合去重、集合操作等场景，不关心顺序和元素重复性。

list的操作

现有一个目标基因的列表，因为分析需求变化需要更改该基因列表。

```
gene_list = ["BRCA1", "EGFR", "TP53", "MYC", "APC"]
```

1. 添加一个新的基因 **PTEN**

```
gene_list.append("PTEN")
```

2. 删除原有基因 **TP53**

```
gene_list.remove("TP53")
```

3. 将基因 **EGFR** 替换成 **ESR1**

```
gene_list[1] = "ESR1"
```

4. 将新的基因表格和原先进行合并 ["KRAS", "BRAF"]

```
New_gene = ["KRAS", "ESR1"]
```

```
All_gene = gene_list + New_gene
```

```
print(All_gene) # ['BRCA1', 'ESR1', 'MYC', 'APC', 'PTEN', 'KRAS', 'ESR1']
```

列表 & 集合 (list & set)

列表(list) 有顺序可变的数据类型, 允许重复元素存在。

```
gene_list = ['BRCA1', 'ESR1', 'MYC', 'APC', 'PTEN', 'KRAS', 'ESR1']
```

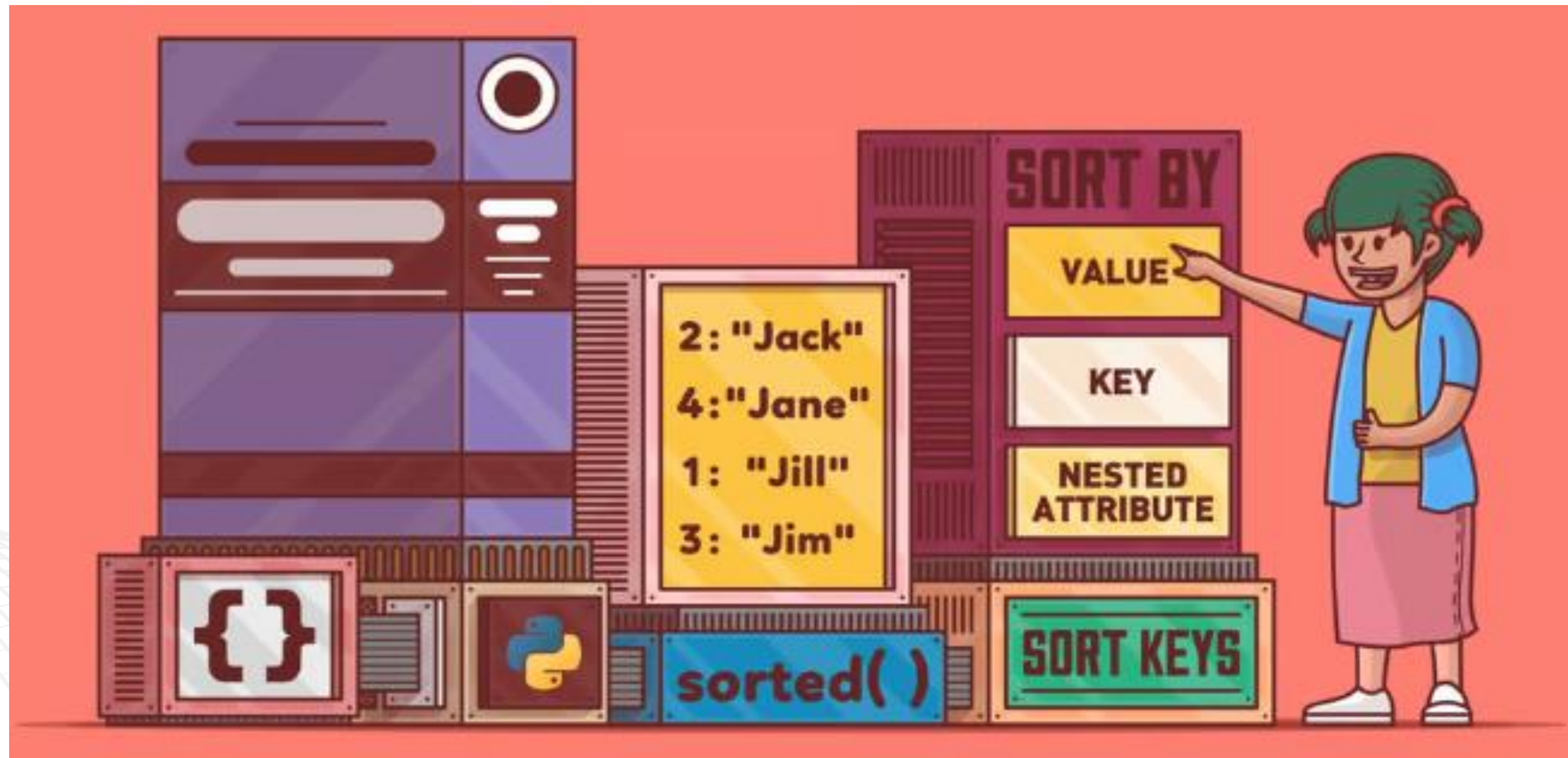
在python中, 'set' 是一种无序且不允许重复元素的数据类型, 它不支持索引和切片, 所以我们可以使用 'set' 来很方便地去除列表中的重复元素。

```
Unique_list = set(gene_list)
```

```
print(Unique_list) # 输出 {'MYC', 'KRAS', 'ESR1', 'PTEN', 'APC', 'BRCA1'}
```


字典 & (dict)

Python字典是一种键值对(key-value pair)的数据结构。字典的每个元素由一个**键**和一个**值**组成，它是可变的,无序的,并且不允许重复的键。在NGS数据分析中,字典常用于存储和处理各种信息。



```
my_dict = {  
    "name": "Alice",  
    "age": 25,  
    "height": "160"  
}  
Name = my_dict["name"]  
print(Name) # Alice  
  
my_dict["city"] = "Suzhou"  
my_dict["age"] = 26  
del my_dict["height"]  
  
# {'name': 'Alice', 'age': 26,  
  'city': 'Suzhou'}
```


字典 (dict)

字典的方法:

```
keys = my_dict.keys() # 获取所有的键
```

```
values = my_dict.values() # 获取所有的值
```

```
items = my_dict.items() # 获取所有的键值对
```

现在有一个储存基因表达水平的字典类型数据

```
gene_expression = { "BRCA1": 15.2, "TP53": 25.7, "EGFR": 42.1, "KRAS": 10.5, "PTEN": 18.3 }
```

```
Keys = gene_expression.keys()
```

```
Values = gene_expression.values()
```

```
Items = gene_expression.items()
```

```
print(Keys) # dict_keys(['BRCA1', 'TP53', 'EGFR', 'KRAS', 'PTEN'])
```

```
print(Values) # dict_values([15.2, 25.7, 42.1, 10.5, 18.3])
```

```
print(Items) # dict_items([('BRCA1', 15.2), ('TP53', 25.7), ('EGFR', 42.1), ('KRAS', 10.5), ('PTEN', 18.3)])
```

字典 (dict)

练习6

请在下列数据中添加基因MYC（它的表达水平为30.8）。并更新TP53的基因表达水平为28.7。基因表达水平（FPKM值）大于20被定义为表达显著的基因，筛选出表达显著的基因？

```
gene_expression = { "BRCA1": 15.2, "TP53": 15.7, "EGFR": 42.1, "KRAS": 10.5, "PTEN": 18.3 }
```

```
gene_expression["MYC"] = 30.8
```

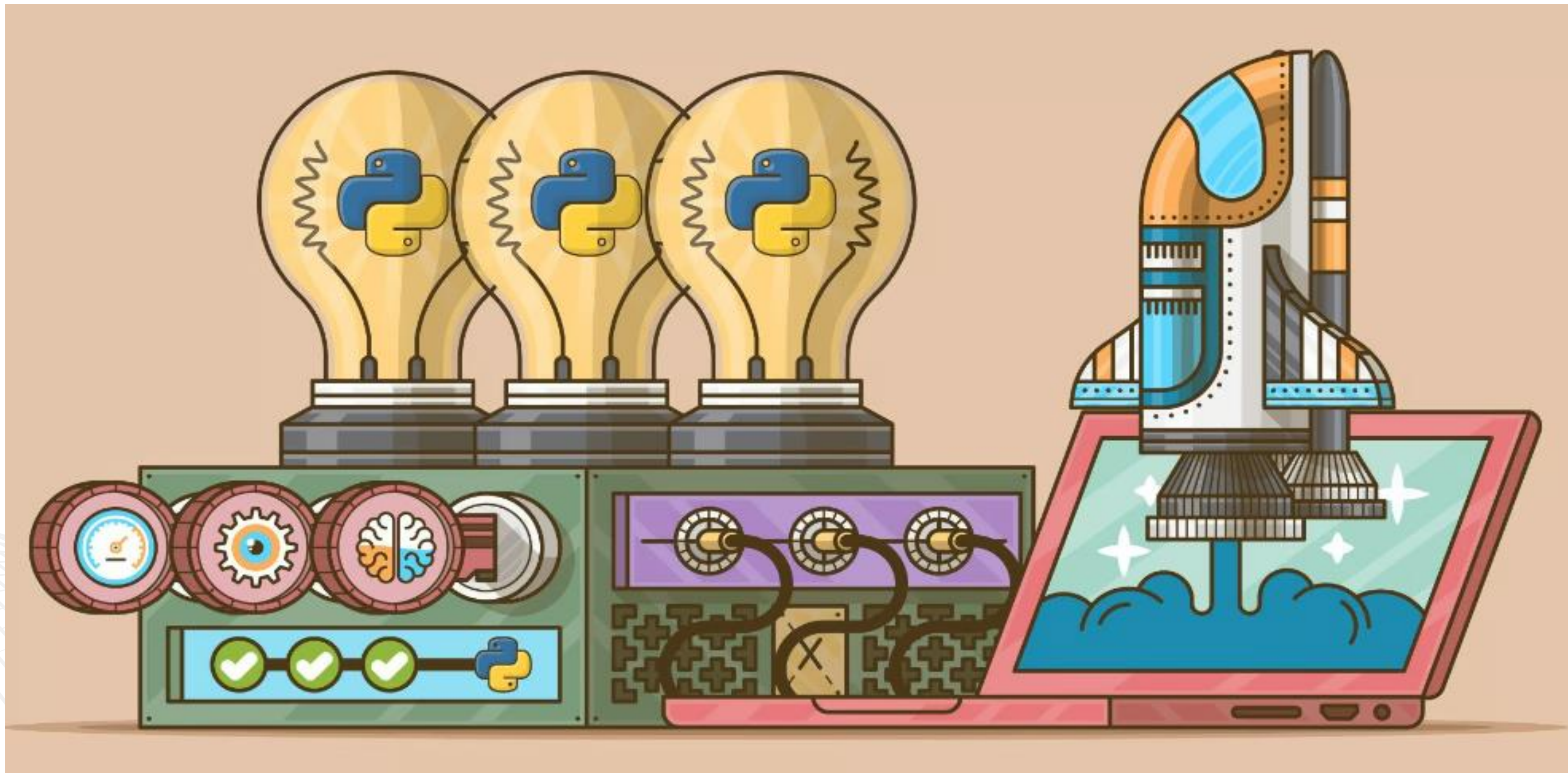
```
gene_expression["TP53"] = 28.7
```

```
print(gene_expression) # {'BRCA1': 15.2, 'TP53': 28.7, 'EGFR': 42.1, 'KRAS': 10.5, 'PTEN': 18.3, 'MYC':  
30.8}
```

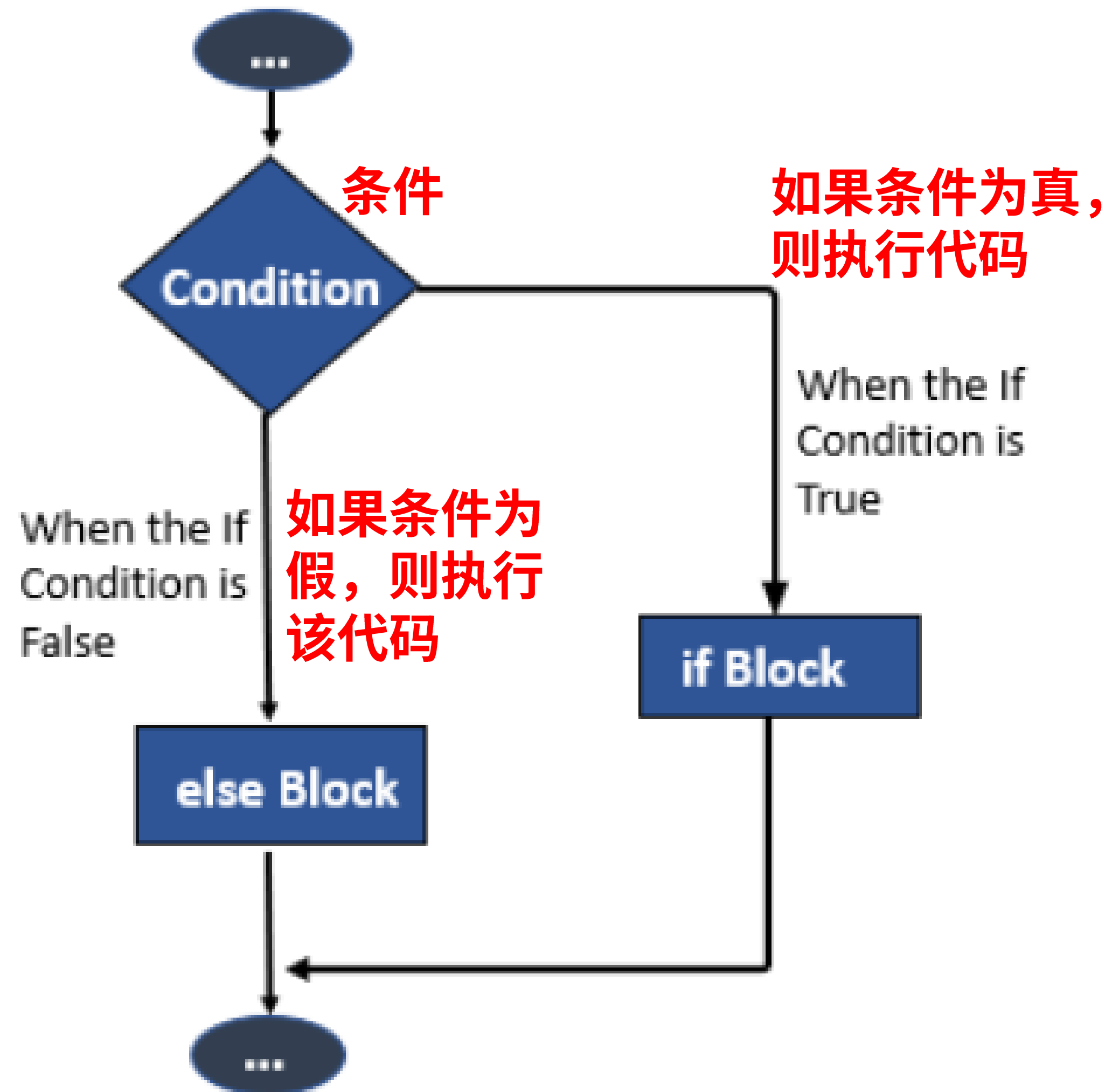
```
print(gene_expression.values()) # dict_values([15.2, 28.7, 42.1, 10.5, 18.3, 30.8])
```


流程控制

流程控制是编程的核心工具，它提供了灵活和强大的手段来管理和组织代码的执行，确保程序能够高效、稳定地运行并适应各种应用场景。Python中的控制流程主要包括条件语句（if-else）、循环语句（for和while）以及异常处理（try-except）。



If-else 条件语句



if 条件表达式
if 判断条件：
 条件为true, 执行语句
else:
 条件为false, 执行语句
以缩进来判断if-else语句的配对情况



循环语句

Python的循环语句，一般情况下是按顺序执行的。循环语句允许我们执行一个语句或语句组多次。

循环类型	描述
For循环	遍历一个可迭代对象，重复执行语句
While循环	判断条件为真执行循环，否则退出循环



流程控制

练习6.1

基因表达水平（FPKM值）大于20被定义为表达显著的基因，筛选出表达显著的基因？

```
gene_expression = { "BRCA1": 15.2, "TP53": 28.7, "EGFR": 42.1, "KRAS": 10.5, "PTEN": 18.3, "MYC": 30.8 }
```

```
for gene in gene_expression.keys():  
    print(f"{gene}")
```

```
# BRCA1  
TP53  
EGFR  
KRAS  
PTEN  
MYC
```

```
for gene, fpkm in gene_expression.items():  
    print(f"{gene}: {fpkm}")
```

```
# BRCA1: 15.2  
TP53: 28.7  
EGFR: 42.1  
KRAS: 10.5  
PTEN: 18.3  
MYC: 30.8
```


流程控制

练习6.2

基因表达水平（FPKM值）大于20被定义为表达显著的基因，筛选出表达显著的基因？

```
gene_expression = { "BRCA1": 15.2, "TP53": 28.7, "EGFR": 42.1, "KRAS": 10.5, "PTEN": 18.3, "MYC": 30.8 }
```

```
threshold = 20
```

```
signature = {}
```

```
for gene, fpkm in gene_expression.items():
```

```
    if fpkm > threshold:
```

```
        signature[gene] = fpkm
```

```
print("\n表达显著基因 (FPKM > 20):")
```

```
for gene, fpkm in signature.items():
```

```
    print(f"{gene}: {fpkm}")
```

遍历gene_expression迭代对象，重复执行语句

表达显著基因 (FPKM > 20):

TP53: 28.7

EGFR: 42.1

MYC: 30.8

流程控制

练习6.3

```
gene_expression = { "BRCA1": 15.2, "TP53": 28.7, "EGFR": 42.1, "KRAS": 10.5, "PTEN": 18.3, "MYC": 30.8 }
```

```
num = 0
```

```
num_gene = len(gene_expression)
```

```
while num < num_gene:
```

```
    gene, fpkm = list(gene_expression.items())[num]
```

```
    if fpkm > threshold:
```

```
        signature[gene] = fpkm
```

```
    num += 1
```

```
print("\n表达显著基因 (FPKM > 20):")
```

```
for gene, fpkm in signature.items():
```

```
    print(f"{gene}: {fpkm}")
```

当num小于基因个数（6）时，重复
执行循环

当num大于基因个数（6）时，结束
循环

```
# num += 1
```

```
num = num + 1
```

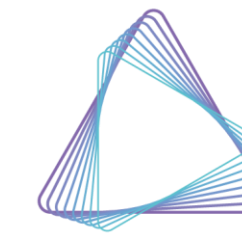
```
# 表达显著基因 (FPKM > 20):
```

```
TP53: 28.7
```

```
EGFR: 42.1
```

```
MYC: 30.8
```


Try-except



AZENTA 安升达
LIFE SCIENCES

try:



运行代码
Run this code

except:



当遇到例外（报错）会执行该代码

Execute this code when
there is an exception

else:



没有例外（报错）？会执行该代码

No exceptions? Run this
code.

Try-except

```
dna_sequence = "ATCG"
try:
    # 尝试获取序列中的第5个碱基（实际上序列只有4个碱基）
    fifth_base = dna_sequence[4]
    print("第5个碱基是:", fifth_base)
except IndexError:
    print("错误：序列长度不足5个碱基")
```


PYTHON常见库介绍

Numpy, Pandas, Matplotlib

03

Python常见库介绍

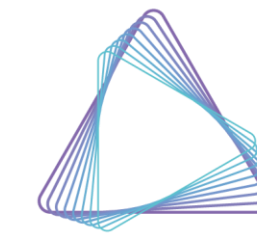


库名	功能	NGS应用领域
Numpy	- 提供多维数组对象和相关工具 - 支持大量的数学函数和数组操作 - 高效的内存使用和计算速度	- 基因表达数据的数值处理 - 序列比对得分的计算 - 测序质量分数的统计分析
Pandas	- 提供 DataFrame 和 Series 数据结构 - 支持处理结构化和表格数据 - 支持数据导入导出（CSV、Excel 等）	- 管理和分析基因表达矩阵 - 处理测序元数据和注释信息 - 差异表达分析结果的组织和过滤
Matplotlib	- 支持各种图表类型（线图、散点图、条形图、直方图等） - 高度可定制化的图表样式 - 支持交互式 and 动态绘图	- 绘制基因表达热图 - 创建测序质量分布图 - 可视化差异表达分析结果

Numpy



NumPy



AZENTA 安升达
LIFE SCIENCES

Numpy是python用于科学计算重要库之一，它提供强大的多维数组对象和用于处理这些数组的函数。

假设我们有10个基因的表达值

导入numpy包 ## pip install numpy 安装包

```
import numpy as np
```

```
gene_expression = np.array([100, 200, 150, 300, 250, 180, 220, 190, 280, 210])
```

计算平均表达值

```
mean_expression = np.mean(gene_expression)
```

```
print(f"\n平均表达值: {mean_expression}")     # 平均表达值: 208.0
```

找出表达值最大的基因

```
max_expression = np.max(gene_expression)
```

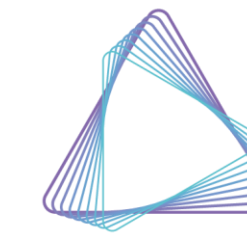
```
max_index = np.argmax(gene_expression)
```

```
print(f"最高表达值: {max_expression}, 索引: {max_index}")     # 最高表达值: 300, 索引: 3
```


Numpy



NumPy



AZENTA 安升达
LIFE SCIENCES

```
gene_expression = np.array([100, 200, 150, 300, 250, 180, 220, 190, 280, 210])
```

对所有基因表达值进行标准化（减去均值再除以标准差）

```
standardized_expression = (gene_expression - np.mean(gene_expression)) / np.std(gene_expression)
```

```
print("\n标准化后的表达值:")
```

```
print(standardized_expression)
```

```
# [-1.91638829 -0.14195469 -1.02917149  1.63247891  0.74526211 -0.49684141  
 0.21293203 -0.31939805  1.27759219  0.03548867]
```

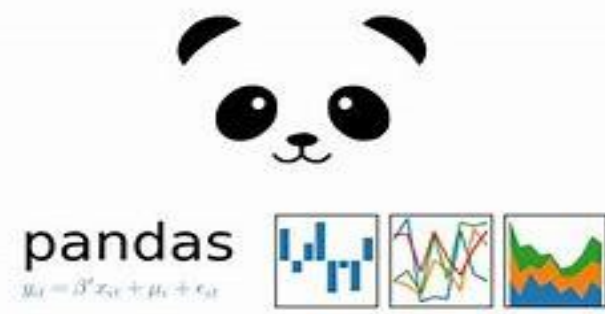
获取表达值大于200的基因

```
high_expressed = gene_expression[gene_expression > 200]
```

```
print("\n高表达基因值:")
```

```
print(high_expressed) #高表达基因值: [300 250 220 280 210]
```

Pandas



Pandas是python用于数据分析和数据处理的重要文库。它建立在Numpy之上，提供了高级数据结构和数据操作工具，使得在Python中进行数据处理变得更加简单，快速和有效。

导入Pandas包

```
import pandas as pd
```

模拟一个基因表达的数据集

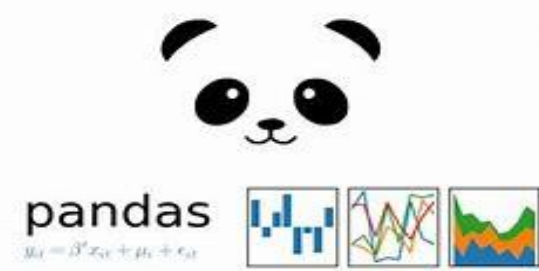
```
data = { 'Gene': ['BRCA1', 'TP53', 'EGFR', 'KRAS', 'PTEN' ],  
'Sample1': [100, 150, 200, 120, 80],  
'Sample2': [110, 140, 190, 130, 90],  
'Sample3': [105, 160, 210, 125, 85] }
```

```
df = pd.DataFrame(data)  
print("基因表达数据集:")  
print(df)
```

基因表达数据集：

	Gene	Sample1	Sample2	Sample3
0	BRCA1	100	110	105
1	TP53	150	140	160
2	EGFR	200	190	210
3	KRAS	120	130	125
4	PTEN	80	90	85

Pandas



`df.info()` # 查看数据信息

`df.describe()` #查看描述性统计

过滤数据

```
high_expressed = df[df['Sample1'] > 150]
```

```
print(high_expressed)
```

添加新列（计算平均表达值）

```
df['Mean_Expression'] = df[['Sample1', 'Sample2',  
'Sample3']].mean(axis=1)
```

```
print(df)
```

我们有另外一个包含基因染色体注释信息的Dataframe，现进行数据合并

```
annotations = pd.DataFrame({ 'Gene': ['BRCA1', 'TP53', 'EGFR',  
'KRAS', 'PTEN'], 'Chromosome': ['17', '17', '7', '12', '10'] })
```

```
merged_df = pd.merge(df, annotations, on='Gene' )
```

```
print(merged_df)
```

将修改过的结果导出CSV文件

```
df.to_csv('gene_expression_data.csv', index=False)
```

```
print("\n数据已导出为CSV文件")
```



AZENTA 安升达
LIFE SCIENCES

在Sample1中高表达的基因：

	Gene	Sample1	Sample2	Sample3
2	EGFR	200	190	210

axis = 1 : 按照列的方向（水平方向）

axis = 0 : 按照行的方向（垂直方向）

添加平均表达值后的数据集：

	Gene	Sample1	Sample2	Sample3	Mean_Expression
0	BRCA1	100	110	105	105.0
1	TP53	150	140	160	150.0
2	EGFR	200	190	210	200.0
3	KRAS	120	130	125	125.0
4	PTEN	80	90	85	85.0

合并注释信息后的数据集：

	Gene	Sample1	Sample2	Sample3	Mean_Expression	Chromosome
0	BRCA1	100	110	105	105.0	17
1	TP53	150	140	160	150.0	17
2	EGFR	200	190	210	200.0	7
3	KRAS	120	130	125	125.0	12
4	PTEN	80	90	85	85.0	10

Matplotlib是Python中最流行的用于数据可视化的库之一。它提供了丰富的绘图功能，可以用于生成各种类型的静态、交互式 and 动态图表。Matplotlib被广泛应用于科学计算、数据分析、工程领域以及学术研究中。

为使图更美观，可以考虑使用Seaborn，它基于Matplotlib构建，提供更美观的默认样式和高级统计图表。

首先导入我们需要的包

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

Matplotlib

读取之前保存的文件

```
df= pd.read_csv('gene_expression_data.csv')
```

设置绘图风格

```
sns.set_style("darkgrid")
```

```
sns.set_palette("Set2")
```

绘制条形图，表示各基因在不同样本中的表达水平

```
plt.figure(figsize=(10, 6)) # 设置图形大小
```

```
df.plot(x='Gene', y=['Sample1', 'Sample2', 'Sample3'], kind='bar')
```

```
plt.title('Gene Expression Across Samples', fontsize=16) #设置主题
```

```
plt.ylabel('Expression Level') #设置y轴标题
```

```
plt.legend(title='Samples') # 设置图注
```

```
plt.tight_layout() # 调整布局
```

```
plt.savefig('gene_expression_bar_chart.png', dpi=300, bbox_inches='tight') # 保存文件
```

```
plt.close() #关闭图形窗口
```

Azenta Life Sciences | Proprietary and confidential.



AZENTA 安升达
LIFE SCIENCES



箱线图： 各基因的表达分布

```
plt.figure(figsize=(10, 6))
```

```
df_melted = df.melt(id_vars=['Gene', 'Mean_Expression'], value_vars=['Sample1', 'Sample2',  
'Sample3'], var_name='Sample', value_name='Expression' )
```

```
sns.boxplot(x='Gene', y='Expression', data=df_melted)
```

```
plt.title('Distribution of Gene Expression', fontsize=16)
```

```
plt.ylabel('Expression Level' )
```

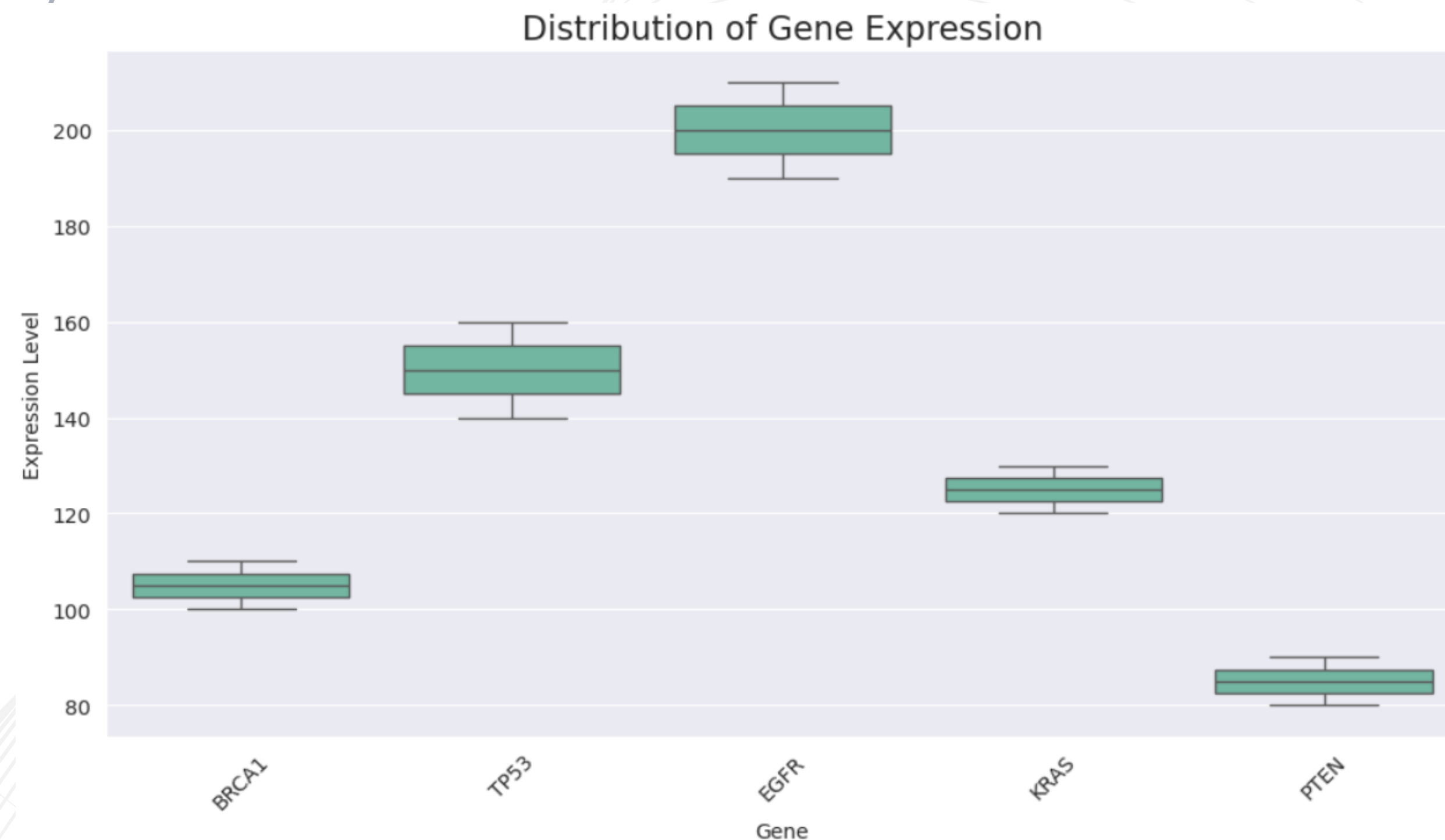
```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

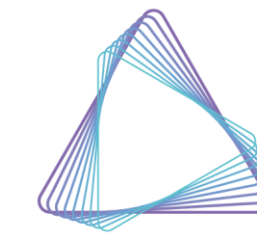
```
plt.savefig('gene_expression_boxplot.png'
```

```
, Dpi=300, bbox_inches= 'tight' )
```

```
Plt.close()
```



Matplotlib



AZENTA 安升达
LIFE SCIENCES

热图：所有样本的表达水平

```
plt.figure(figsize=(10, 6))
```

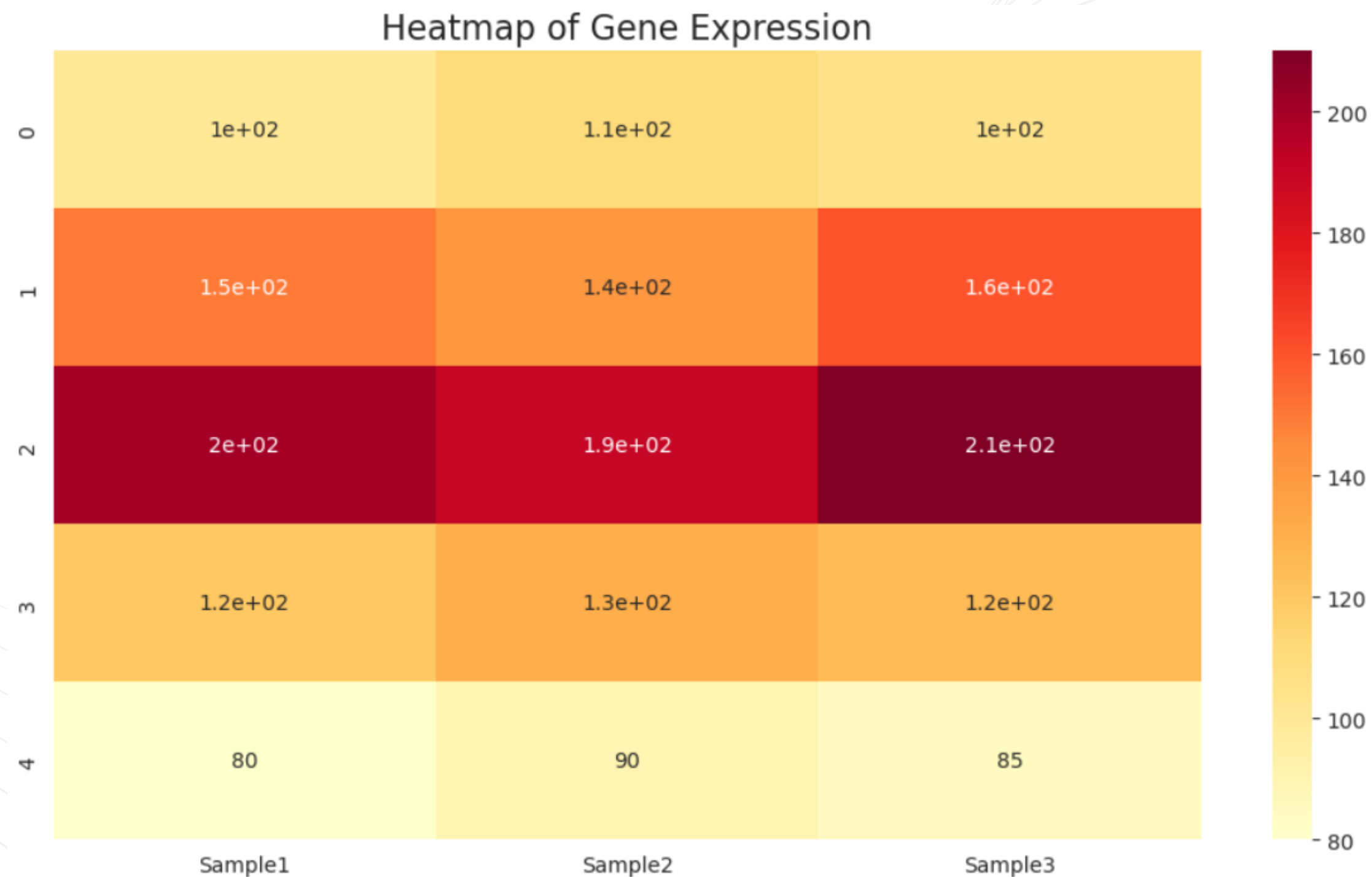
```
sns.heatmap(df[['Sample1', 'Sample2', 'Sample3']], annot=True, cmap='YlOrRd')
```

```
plt.title('Heatmap of Gene Expression', fontsize=16)
```

```
plt.tight_layout()
```

```
plt.savefig('gene_expression_heatmap.png', dpi=300, bbox_inches='tight' )
```

```
plt.close()
```



BIOPYTHON

介绍和用法

04



Biopython

Biopython是一个用于生物信息学的Python库，它提供了一系列用于处理生物学数据的工具和模块。Biopython包括各种生物信息学文件格式的解析器（BLASTA, Clustalw, FASTA, Genbank等），访问NCBI, Expasy等线上数据库，以及序列比对，基序分析工具，聚类算法和结构生物学模块等。

Biopython的安装和使用方法

与其他python包相同的安装方法：

```
>>> pip install biopython
```

使用方法：

```
>>> import Bio
```

查看版本信息：

```
>>> print(Bio.__version__)
```

Biopython

Biopython支持解析多种生物信息学文件，包括：

- Blast output – both from standalone and WWW Blast
- Clustalw
- FASTA
- GenBank
- PubMed and Medline
- ExPASy files, like Enzyme and Prosite
- SCOP, including ‘dom’ and ‘lin’ files
- UniGene
- SwissProt

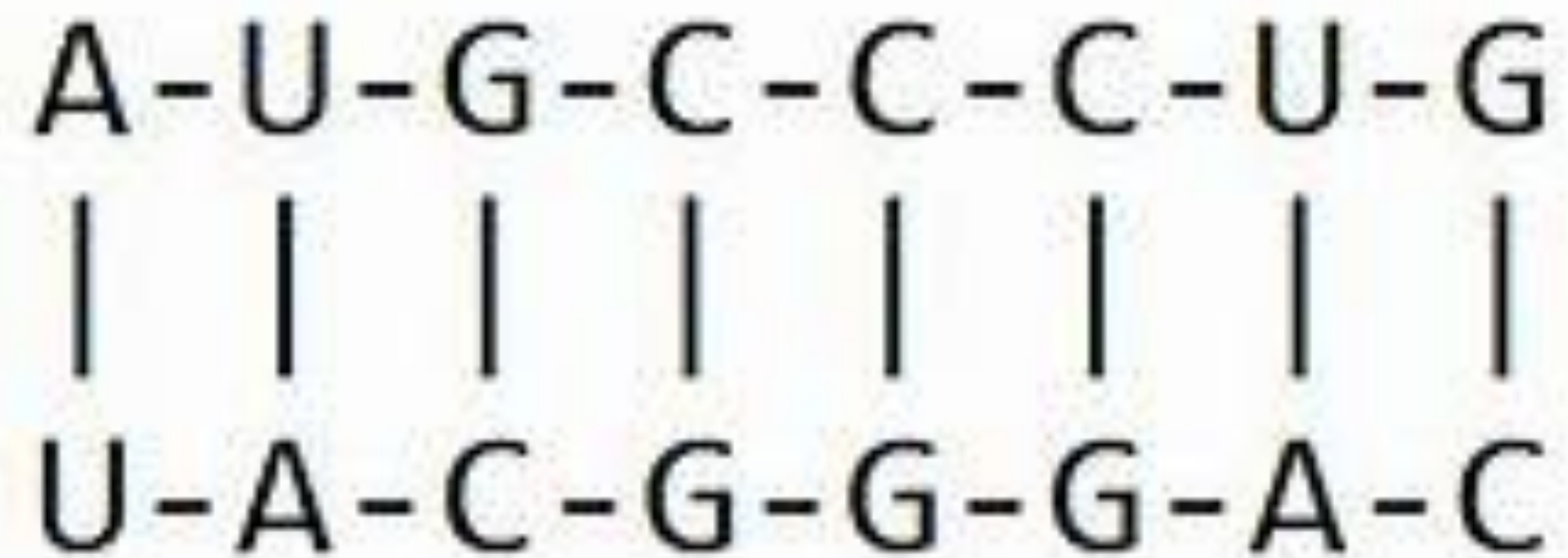
示例

```
from Bio import SeqIO
genbank_file = "example.gb"
records =
SeqIO.parse(genbank_file, "genbank")

for record in records:
    print(record.id)
    print(record.name)
    print(record.description)
    print(record.seq)
```

序列处理

我们接下使用biopython来处理DNA序列信息，使用biopython来快速得到该序列的反向互补序列。



A-U-G-C-C-C-U-G
| | | | | | | |
U-A-C-G-G-G-A-C

序列处理

我们接下使用biopython来处理DNA序列信息，使用biopython来快速得到该序列的反向互补序列。

```
from Bio.Seq import Seq
```

```
my_seq = Seq("AGTACACTGGT")
```

互补序列

```
my_seq.complement() # Seq('TCATGTGACCA')
```

反向序列

```
my_seq.reverse_complement() # Seq('ACCAGTGTACT')
```

FASTA文件的解析

我们可以使用biopython对fasta文件进行快速解析。

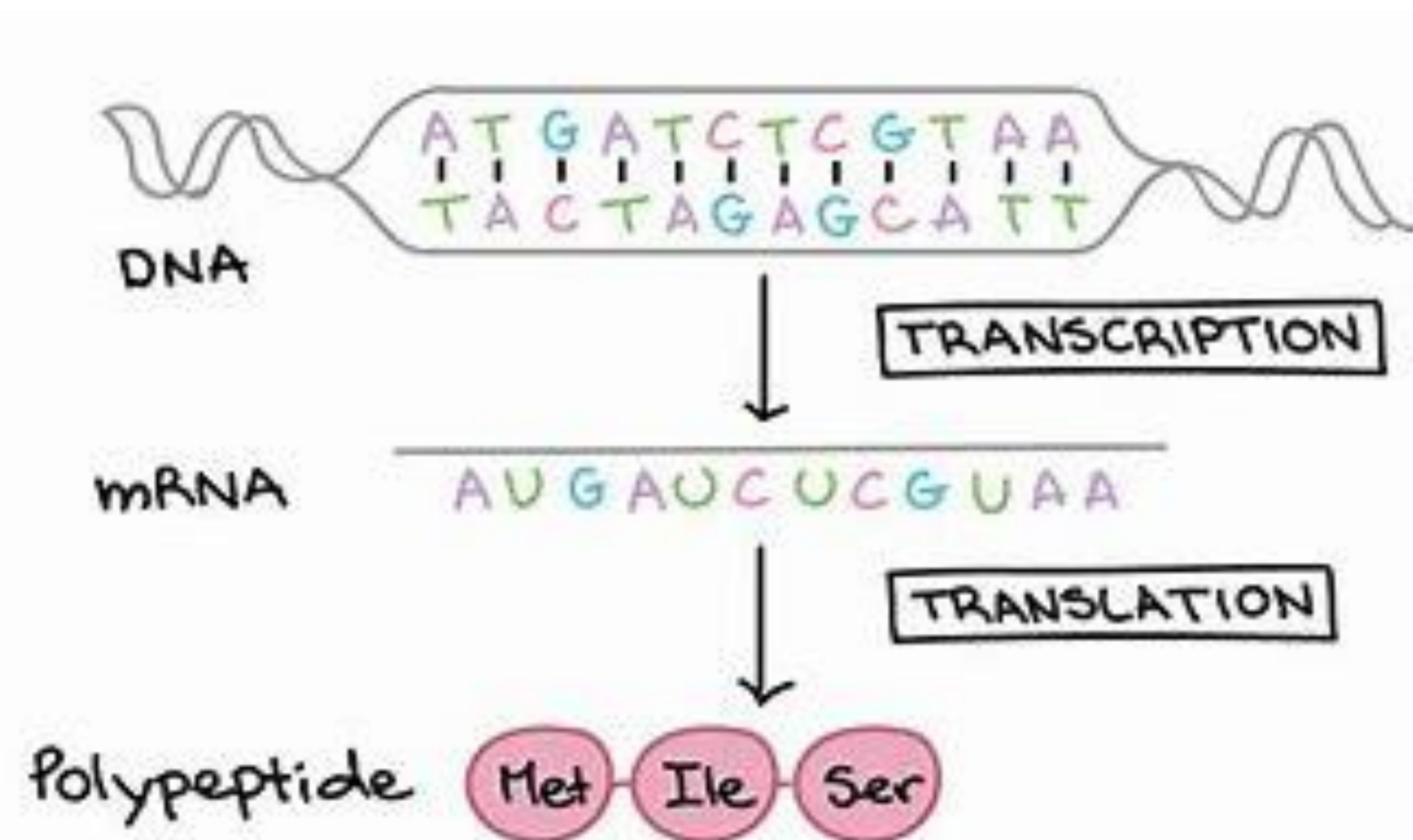
```
from Bio import SeqIO

# 读取FASTA文件
fasta_sequences = SeqIO.parse(open("example.fasta"), "fasta")

for fasta in fasta_sequences:
    name, sequence = fasta.id, str(fasta.seq)
    print(f"> {name}\n{sequence}")
```

转录翻译

使用Biopython进行转录（将DNA转化为mRNA）和翻译（将mRNA翻译为蛋白质）。



```
from Bio.Seq import Seq
coding_dna = Seq("ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG")
```

```
messenger_rna = coding_dna.transcribe()
print(messenger_rna) # AUGGCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG
```

```
Polypeptide = gene.translate(table="Bacterial")
print(Polypeptide) # VKKMQSIVLALSLVLVAPMA
```


BIOPYTHON计算GC含量

使用biopython计算序列的GC含量。

GC含量的计算公式

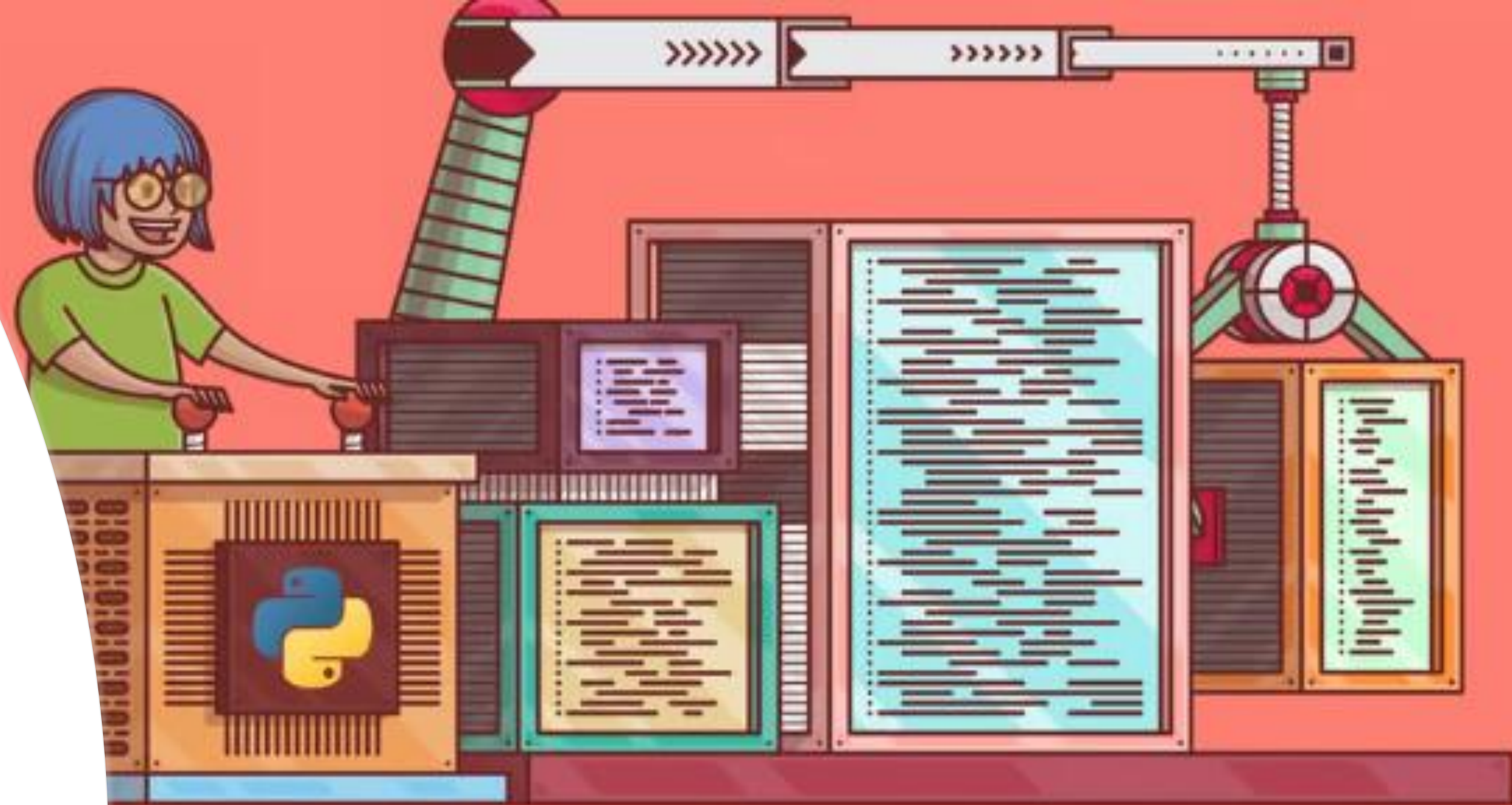
$$\text{GC含量(\%)} = \left(\frac{G+C}{A+T+G+C} \right) \times 100$$

```
from Bio.Seq import Seq  
from Bio.SeqUtils import gc_fraction
```

```
my_seq = Seq("GATCGATGGGCCTATATAGGATCGAAAATCGC")  
GC = gc_fraction(my_seq) * 100  
print(GC) # 46.875
```

函数

- 函数是Python编程中的一个核心概念,它们允许我们组织代码、提高代码的可重用性和可读性。
- 函数是一组可重用的代码,用于执行特定的任务。它可以接受输入(参数),执行一些操作,并返回结果。
- Python中使用def关键字来定义函数



```
def function_name(parameters):  
    """文档字符串: 描述函数的功能"""  
    # 函数体  
    return result
```


函数

使用biopython计算序列的GC含量。

GC含量的计算公式

$$\text{GC含量(\%)} = \left(\frac{G+C}{A+T+G+C} \right) \times 100$$

```
from Bio.Seq import Seq
from Bio.SeqUtils import gc_fraction
```

```
my_seq = Seq("GATCGATGGGCCTATATAGGATCGAAAATCGC")
GC = gc_fraction(my_seq) * 100
print(GC) # 46.875
```

```
def gc_content(seq):
    gc_count = seq.count('G') + seq.count('C')
    return gc_count / len(seq) * 100
```

```
gc = gc_content(my_seq)
print(gc) # 46.875
```


模仿python的内置函数sum，返回列表的总和？

`def my_sum(numbers):` **## 参数： numbers为输入列表 返回： 列表中所有数值之和**

`total = 0`

for num **in** numbers:

`total = total + num`

return total

`read_depths = [30, 45, 28, 50, 65, 42, 38, 55, 60, 48]`

`Total = my_sum(read_depths)`

`print(Total)` **# 461**

`print(sum(read_depths))` **# 461**

函数

练习 7

模仿python的内置函数len，返回列表的总长？

`def my_len(items):` # 参数: items 为输入列表 返回: 列表的元素总个数

`count = 0`

`for item in items:`

`count = count + 1`

`return count`

`read_depths = [30, 45, 28, 50, 65, 42, 38, 55, 60, 48]`

`print(my_len(read_depths))` # 10

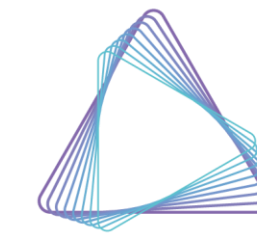
`print(len(read_depths))` # 10

PYTHON进行NGS数据分析

案例分析

05

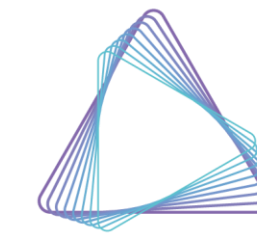
实战：从GenBank文件中提取基因序列并进行分析



AZENTA 安升达
LIFE SCIENCES

生信工程师小明收到一个GenBank文件example.gb, 其中包含了完整的基因组信息。他需要从中提取基因序列并进行分析。提取所有编码区（CDS）的基因序列, 并计算每个基因的GC含量，并绘制GC含量分布图。

实战：从GenBank文件中提取基因序列并进行分析



AZENTA 安升达
LIFE SCIENCES

加载所需要的包

```
from Bio import SeqIO
```

```
from Bio.SeqUtils import gc_fraction
```

```
import matplotlib.pyplot as plt
```

指定genbank文件路径

```
genbank_file = "example.gb "
```

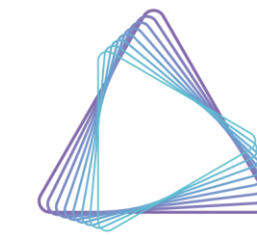
使用SeqIO读取genbank格式文件

```
record = SeqIO.read(genbank_file, "genbank")
```

自定义空的列表，用于存储数据

```
gc_contents = []
```

实战：从GenBank文件中提取基因序列并进行分析



AZENTA 安升达
LIFE SCIENCES

遍历genebank文件中的feature列，筛选feature类型为'CDS'的数据集

```
for feature in record.features:
```

```
    if feature.type == "CDS":
```

提取类型为CDS的所有序列

```
        gene_seq = feature.extract(record.seq)
```

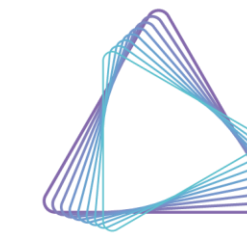
计算序列的GC含量，并将计算所得的值存储到我们之前创建的空列表中

```
        gc_contents.append(gc_fraction(gene_seq))
```

计算平均GC含量

```
average_gc = sum(gc_contents)/len(gc_contents)
```


实战：从GenBank文件中提取基因序列并进行分析



AZENTA 安升达
LIFE SCIENCES

绘制GC含量分布图

使用hist函数绘制直方图，bins划分20个区间

```
plt.hist(gc_contents, bins=20, edgecolor='black' )
```

设置x轴标签

```
plt.xlabel("GC Content (%)")
```

设置y轴标签

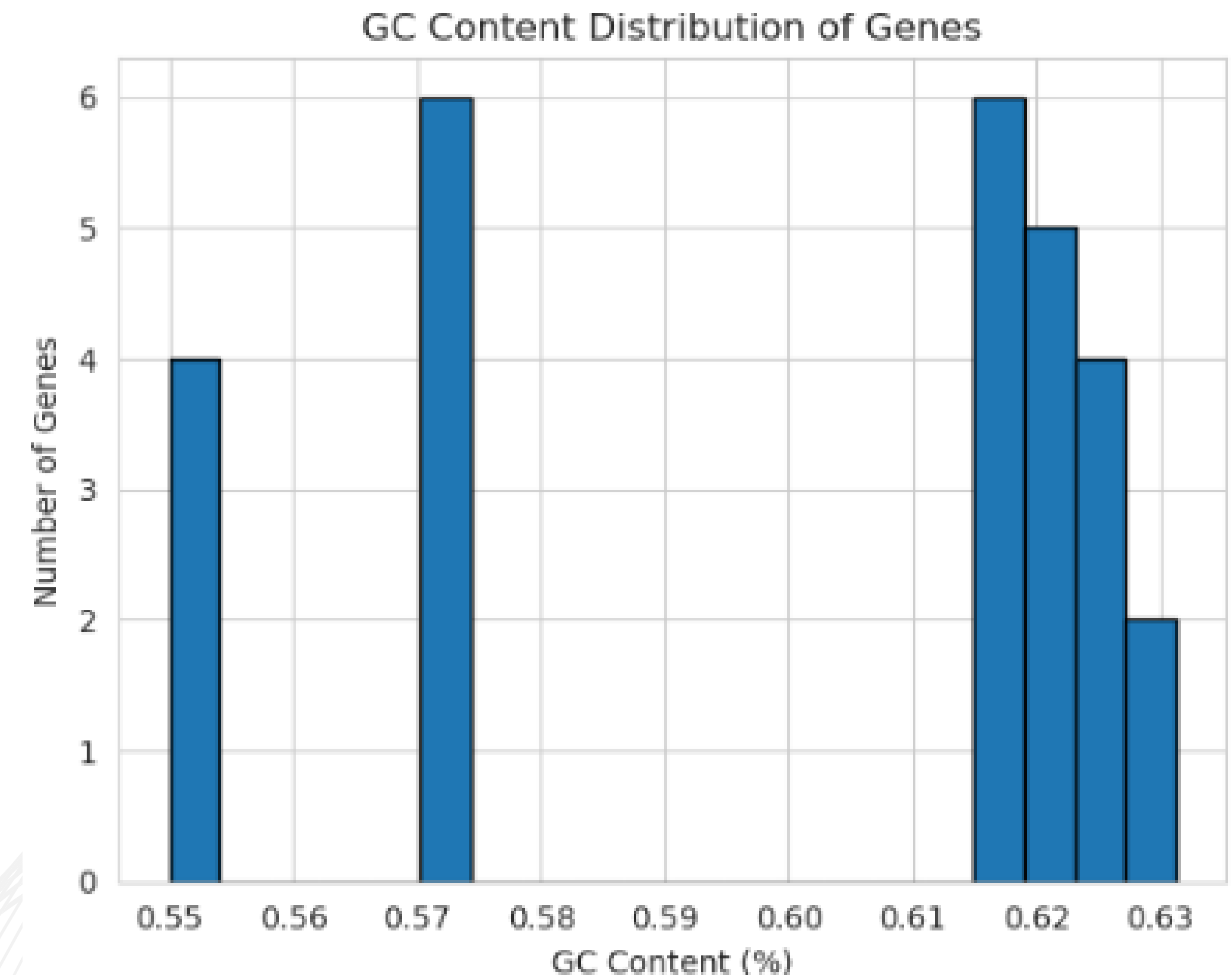
```
plt.ylabel("Number of Genes")
```

设置标题

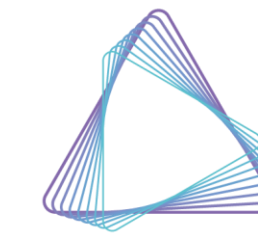
```
plt.title("GC Content Distribution of Genes")
```

创建图形

```
plt.show()
```

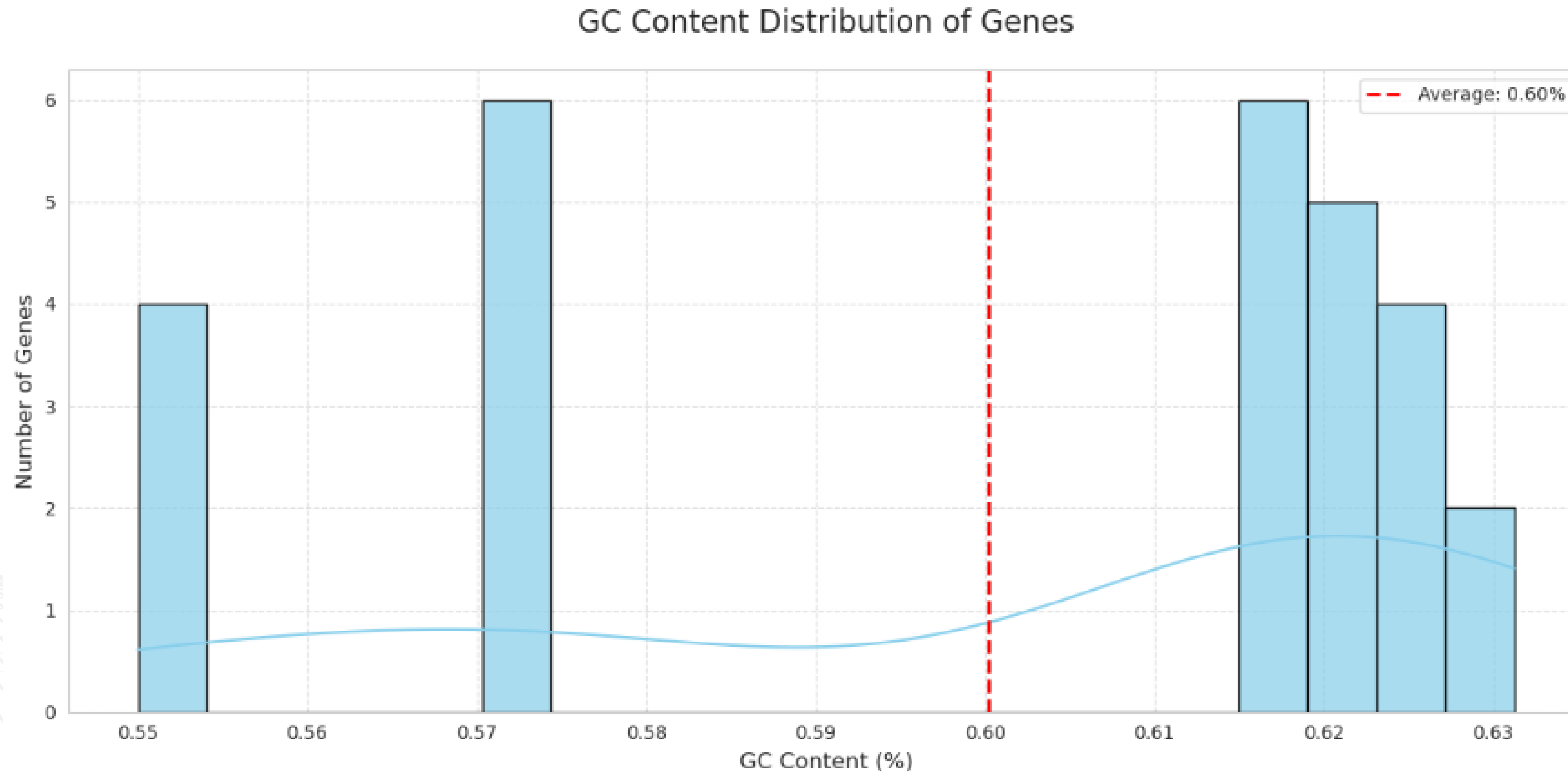


实战：从GenBank文件中提取基因序列并进行分析



AZENTA 安升达
LIFE SCIENCES

通过seaborn以及参数调节，也可以获得更美观的图形。



实战：差异表达基因分析与可视化

生信工程师小明得到了一组差异表达分析结果。这些结果包含了基因ID、基因名称、log2 fold change、p值和校正后的p值(padj)。现在,需要对这些数据进行处理、分析和可视化。

任务:

- 1.读取并处理数据
- 2.筛选显著差异表达的基因
- 3.进行基本的统计分析
- 4.创建一个火山图来可视化差异表达基因
- 5.对top 20的上调和下调基因进行热图绘制

实战：差异表达基因分析与可视化

导入所需要的包

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

读取数据

```
df = pd.read_csv("diff_expression_results.csv")
```

查看数据基本信息

```
print(df.info())
```

```
print(df.head())
```

筛选显著差异表达基因

```
sig_genes = df[(abs(df['log2FoldChange']) > 1) & (df['padj'] < 0.05)]
```

实战：差异表达基因分析与可视化

计算上调和下调基因数量

```
up_regulated = sig_genes[sig_genes['log2FoldChange'] > 0]
```

```
down_regulated = sig_genes[sig_genes['log2FoldChange'] < 0]
```

```
print(f"上调基因数量: {len(up_regulated)}")
```

```
print(f"下调基因数量: {len(down_regulated)}")
```

计算log2FoldChange的平均值和标准差

```
mean_log2fc = np.mean(sig_genes['log2FoldChange'])
```

```
std_log2fc = np.std(sig_genes['log2FoldChange'])
```

```
print(f"log2FoldChange平均值: {mean_log2fc:.2f}")
```

```
print(f"log2FoldChange标准差: {std_log2fc:.2f}")
```


实战：差异表达基因分析与可视化

```
colors = ['blue' if x < -1 and y < 0.05 else 'red' if x > 1 and y < 0.05 else 'gray' for x, y in  
zip(df['log2FoldChange'], df['padj'])]
```

```
plt.figure(figsize=(10, 8))
```

```
plt.scatter(df['log2FoldChange'], -np.log10(df['padj']), c=colors, alpha=0.5)
```

```
plt.xlabel('log2 Fold Change')
```

```
plt.ylabel('-log10 adjusted p-value')
```

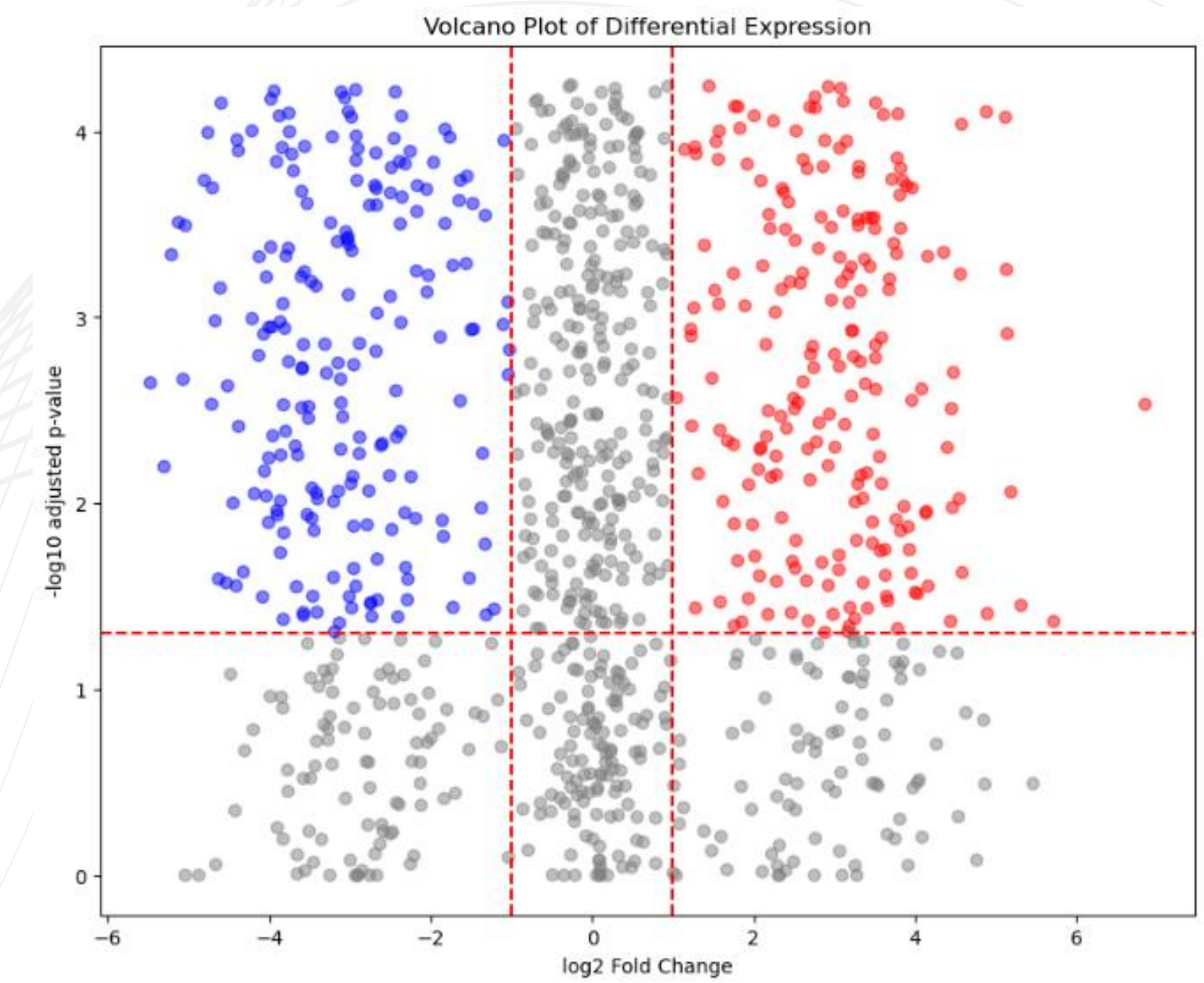
```
plt.title('Volcano Plot of Differential Expression')
```

```
# 添加显著性阈值线
```

```
plt.axhline(y=-np.log10(0.05), color='r', linestyle='--')
```

```
plt.axvline(x=1, color='r', linestyle='--')
```

```
plt.axvline(x=-1, color='r', linestyle='--')plt.show()
```



实战：差异表达基因分析与可视化

```
import seaborn as sns
```

整合上下调top20的基因

```
top_up = up_regulated.nsmallest(20, 'padj')
```

```
top_down = down_regulated.nsmallest(20, 'padj' )
```

concat函数可将多个表格数据合并成一个

```
top_genes = pd.concat([top_up, top_down])
```

绘制热图

```
plt.figure(figsize=(12, 10))
```

#cmap为控制颜色参数，center控制颜色映射的中心值， yticklabels Y轴的标签显示方式

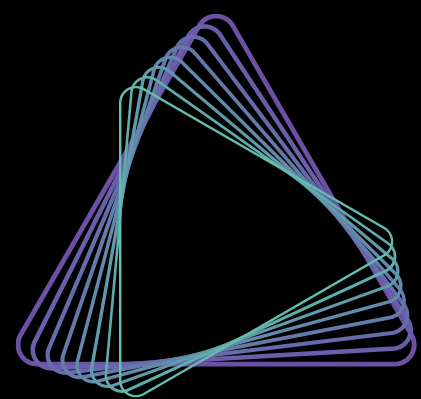
```
sns.heatmap(top_genes[['log2FoldChange']], cmap='RdBu_r', center=0, yticklabels=top_genes.gene_id)
```

```
plt.title('Top 40 Differentially Expressed Genes')
```

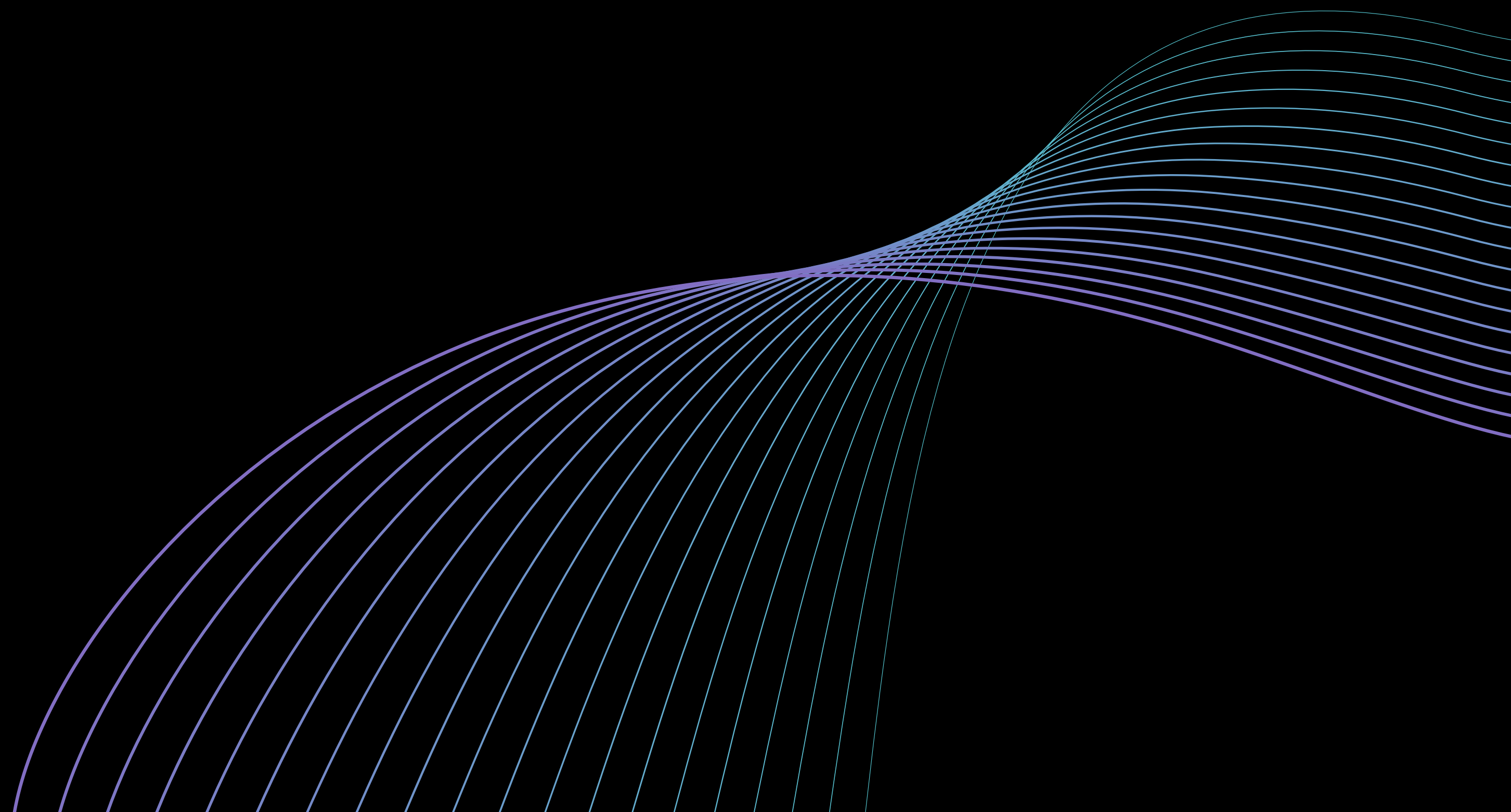
```
plt.show()
```

‘**nsmallest**’ 为函数，用于可迭代对象中找到最小的 n 个元素。

Thank you!



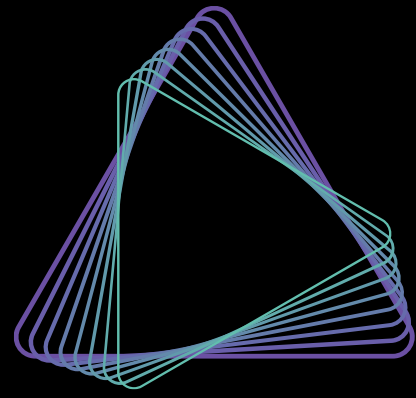
AZENTA 安升达
LIFE SCIENCES



参考链接

[Python 官方文档](#)

[Python 入门教程](#)



AZENTA 安升达
LIFE SCIENCES

练习题

练习1

题目：转换fastq文件的ID

背景：取一个FASTQ文件，并将其中每个序列的ID都修改为统一格式：SEQ_ID1, SEQ_ID2，依此类推，修改后的内容应保存为原FASTQ格式的文件。

任务：编写一个Python脚本，实现以下功能：

- 1.读取指定的FASTQ文件。
- 2.更改ID信息。
- 3.保存原文件。

练习1

题目：转换fastq文件的ID

背景：取一个FASTQ文件，并将其中每个序列的ID都修改为统一格式：SEQ_ID1, SEQ_ID2，依此类推，修改后的内容应保存为原FASTQ格式的文件。

```
@A00202:1652:HCTLWDSXC:2:1101:1814:1000 1:N:0:TAAGGCGA+TAGCTTGC
GGATATATTGTGGGCGATGTGCGCTCTGCCCACTGACGGGCACCGGAGCCAATTCCCACCTCCTTTCAAGACCT
+
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF,FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
```

```
>SEQ_ID1
GGATATATTGTGGGCGATGTGCGCTCTGCCCACTGACGGGCACCGGAGCCAATT
+
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF,FF
>SEQ_ID2
TTAGTTCTTGTGGAAAGGACGAAACACCGACGTCCTCCCACCCACACAGGTTT
+
FFFFFFF:FFFFFFF:FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
```


练习2

题目：FASTQ到FASTA格式转换器

背景： FASTQ和FASTA是两种常用的生物序列文件格式。FASTQ文件包含序列数据及其质量分数，而FASTA文件只包含序列数据。在某些分析中，我们可能只需要序列信息，因此需要将FASTQ文件转换为FASTA文件。

任务： 编写一个Python脚本，实现以下功能：

- 1.读取指定的FASTQ文件。
- 2.提取每条序列的ID和序列信息。
- 3.将提取的信息以FASTA格式写入新文件。

练习2



题目：FASTQ到FASTA格式转换器

背景： FASTQ和FASTA是两种常用的生物序列文件格式。FASTQ文件包含序列数据及其质量分数，而FASTA文件只包含序列数据。在某些分析中，我们可能只需要序列信息，因此需要将FASTQ文件转换为FASTA文件。

```
@SEQ_ID1
GATTTGGGGTTCAAAGCAGTATCGATCAAATAGTAAATCCATTTGTTCAACTCACAGTTT
+
! ' '*((( (**+))%%%++) (%%%) .1***-+*' '))*55CCF>>>>>CCCCCCCC65
@SEQ_ID2
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATC
+
! ' '*((( (**+))%%%++) (%%%) .1***-+*' '))*55CCF>>>>>CCCCCCCC65
```

```
>SEQ_ID1  
GATTTGGGGTTCAAAGCAGTATCGATCAATAGTAAATCCATTTGTTCACACTCACAGTTT  
  
>SEQ_ID2  
ATCGATCGATCGATCGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCGATCGATCGATCGATCGATCGATCATGC
```

练习3

题目：FASTA文件分割

背景：将FASTA序列分割成指定长度的短序列。

任务：编写一个Python脚本，实现以下功能：

- 1.读取指定的FASTA文件。
- 2.按照指定的长度分割序列信息。
- 3.将分割的信息以FASTA格式写入新文件。

练习3

题目：FASTA文件分割

背景：将FASTA序列分割成指定长度的短序列。

```
>fragment_1;random_sequence_1000_bases_start=1;end=100;length=100;source_length=1000
gtttcaaactgtccaggtctaattggtgtgaagcgatactgtttgatgcctgggttggtgca
actggcggcaaaacggcttaaagctccagatgccacatta

>fragment_2;random_sequence_1000_bases_start=101;end=200;length=100;source_length=1000
gtgaggacctgggtgtattacgttggattaccctgtgacagctcaggcacacaccaggatt
aggaaattagcatcgtaagactagtcctgtacaaacaaaaa

>fragment_3;random_sequence_1000_bases_start=201;end=300;length=100;source_length=1000
caaaaaatagaagtttcaagtgggtcccggttggtggagcgcgtaagggatacgtaataat
aggtcaggtcagaaagaagcggcttgagagcccccatttata

>fragment_4;random_sequence_1000_bases_start=301;end=400;length=100;source_length=1000
tctttaacatcgggagaatcatccgtcgaccttggtggcacgggttgccggcgcggaatagg
gtggacgtagatattagacggagacgggttgaagcttacct
```

练习4

题目：DNA碱基统计

背景：统计序列中所有碱基出现的次数以及出现的频率

任务：编写一个Python脚本，实现以下功能：

- 1.读取指定的FASTA文件。
- 2.统计所有碱基（A,T,C,G,N）的出现次数以及它们的频率。
- 3.输出统计结果。

练习5

题目：CRISPR目标序列提取

背景：现有一个CRISPR项目的数据，从FASTA文件中读取序列，并根据给定的上游和下游序列提取sgRNA目标序列。

任务：编写一个Python脚本，实现以下功能：

1. 读取指定的FASTA文件。
2. 提取目标序列。
3. 输出保存结果文件。

练习5

题目：CRISPR目标序列提取

背景：现有一个CRISPR项目的数据，从FASTA文件中读取序列，并根据给定的上游和下游序列提取sgRNA目标序列。

```
@A00202:1652:HCTLWDSXC:2:1101:10239:1000 1:N:0:TAAGGCGA+TAGCTTGC
CGATAATCTTGTGGAAAGGACGAAACACCGGAAGTGCACTCACTTGCTGTTT TAGAGCTAGAAA
+
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
@A00202:1652:HCTLWDSXC:2:1101:11541:1000 1:N:0:TAAGGCGA+TAGCTTGC
TTCGTCTTGTGGAAAGGACGAAACACCGATTGGAGTGGACTTCGAGGTTT TAGAGCTAGAAATA
```